



UNIVERSITÀ
di **VERONA**

PHD School
in **NATURAL SCIENCES
AND ENGINEERING**

Brain Computer Interfaces

Laboratory

Dr. Lorenza Brusini

Dept. of Computer Science, University of Verona

email: lorenza.brusini@univr.it

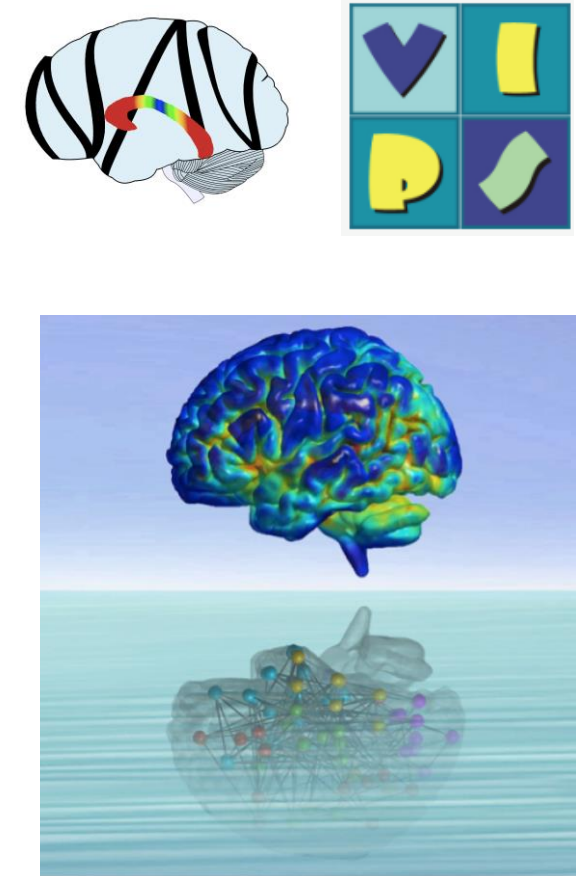
You can find me at @

First floor Ca' Vignal 2, room 1.64B

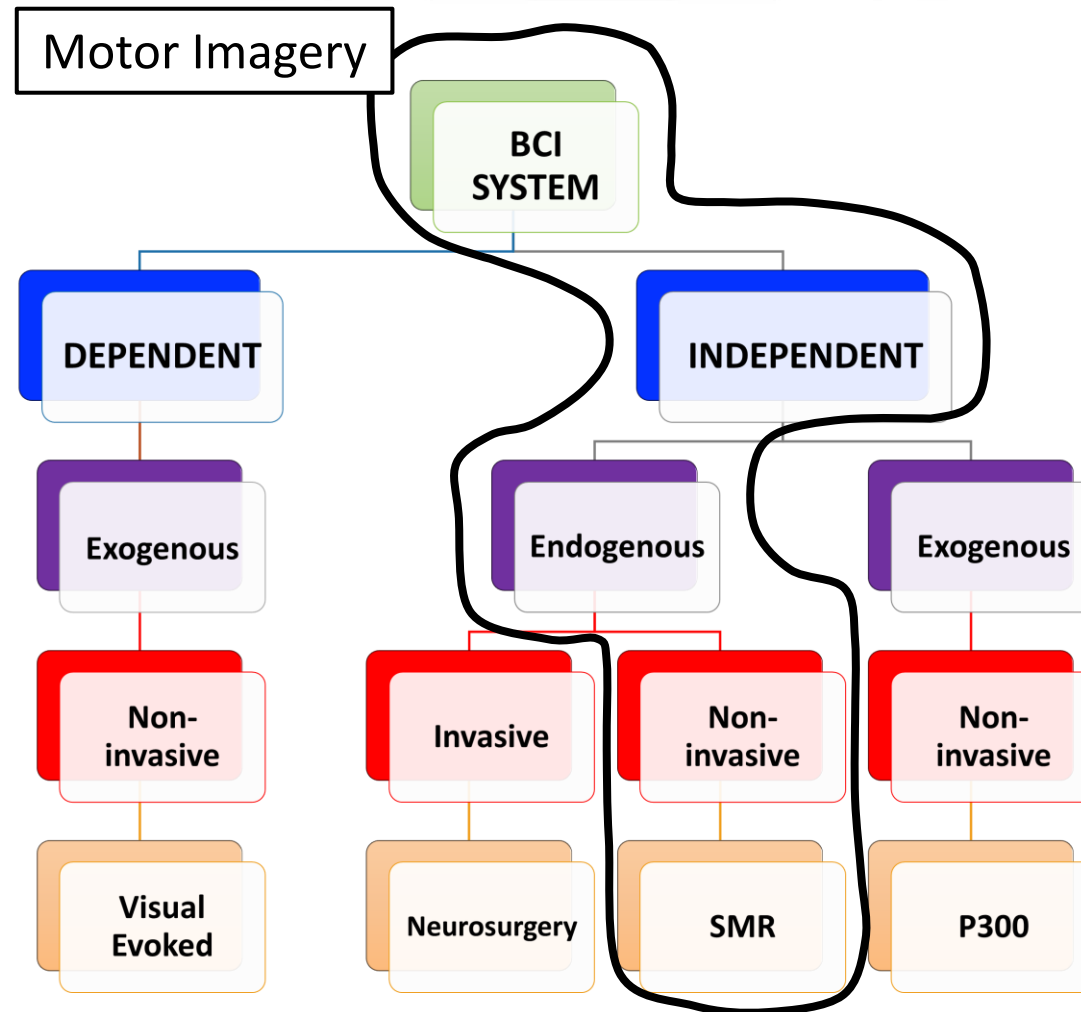
Neuroimaging Lab (CV2 piano -1)

Vision, Image Processing & Sound Lab, VIPS (CV2 piano -2)

<http://vips.sci.univr.it>



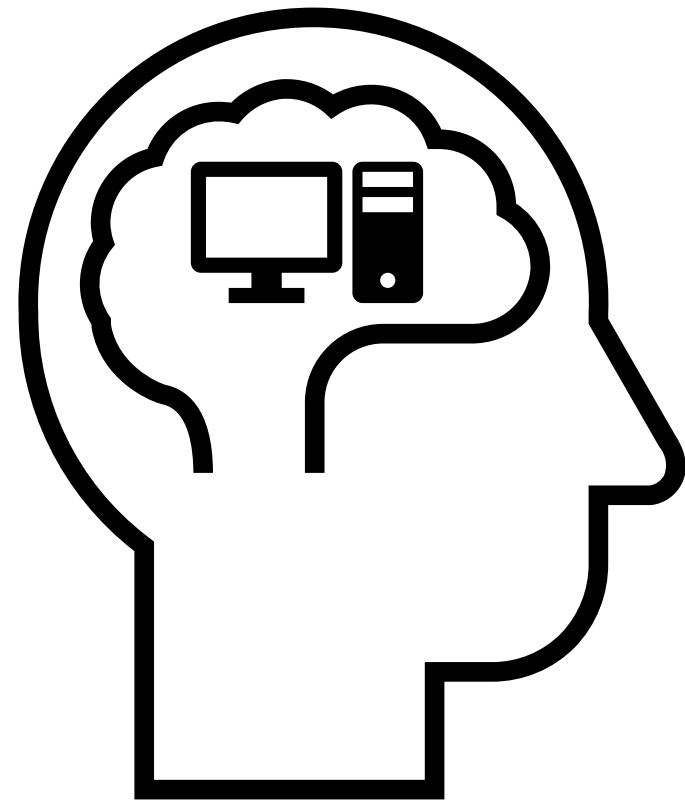
Approach of the laboratory



Approach of the laboratory

The lectures target at:

- Learning from code
- Giving some necessary theoretical details
- Practicing on the code using real data
 - Data recorded in the context of the BCI Competition will be used along the course (publicly available at: <http://www.bbc.de/competition/>)

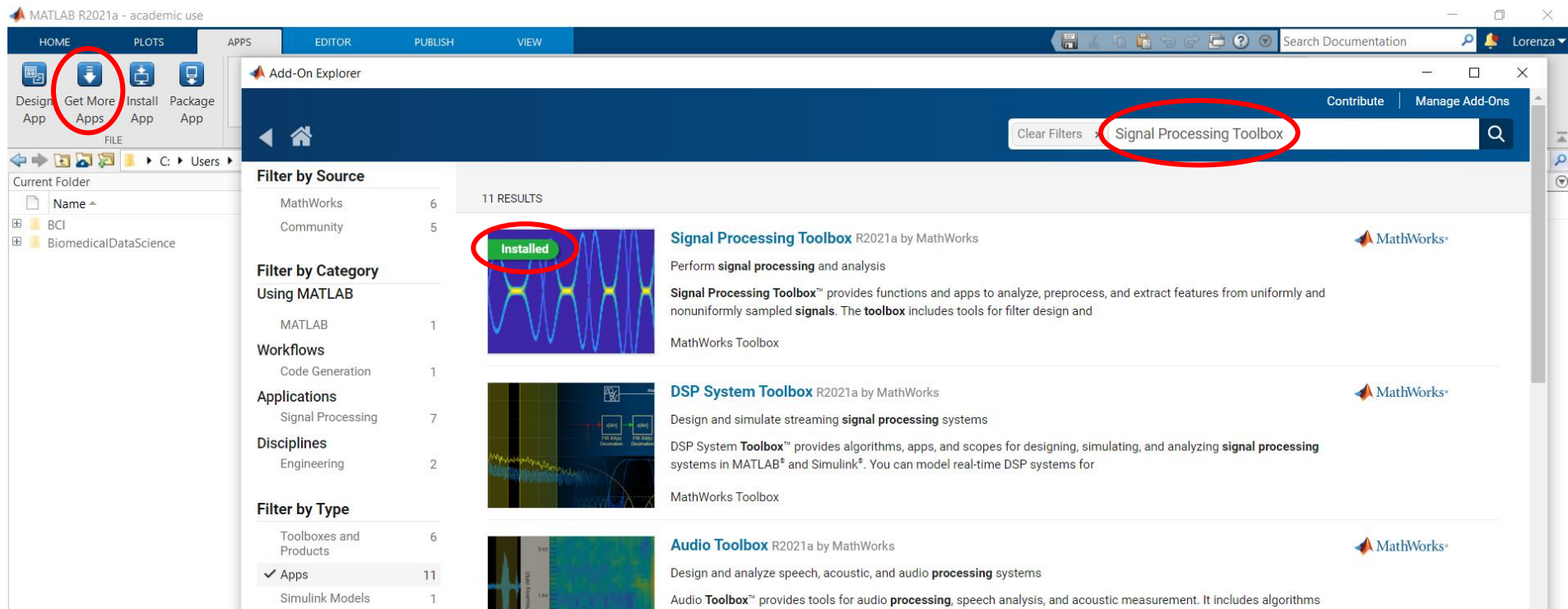


Final evaluation

- On the 9th of September, part of the lecture will be dedicated to the final evaluation
- The final evaluation will consist in the analysis of a new data set by using the notions learned along the course
- The new data set for the final evaluation will be always provided by the public database stored in the context of the BCI Competition.
- However,
 - Fantasy is welcome!
 - If desired, own data set can be used!

Tools

- The course will be based on MATLAB software and two of the MATLAB add-on toolboxes:
 - Signal Processing Toolbox
 - Statistics and Machine Learning Toolbox



Tools

- Moreover, the MATLAB toolbox EEGLAB will be used

Download and installation:

1. Provided at: <https://sccn.ucsd.edu/eeglab/downloadtoolbox.php>
2. Unzip the EEGLAB zip file in the folder of your choice
3. Start Matlab
4. Add the EEGLAB folder just uncompressed to the MATLAB path

```
addpath('eeglab2021.1')
```



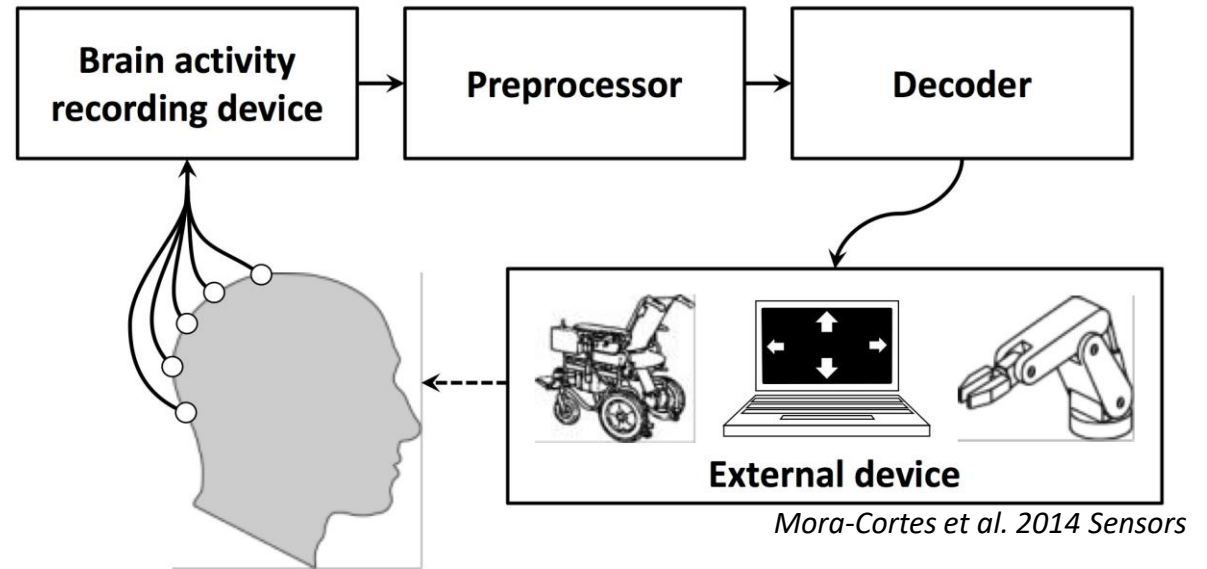
Let's start...

For any question, just ask or type in the ZOOM chat!

The BCI model

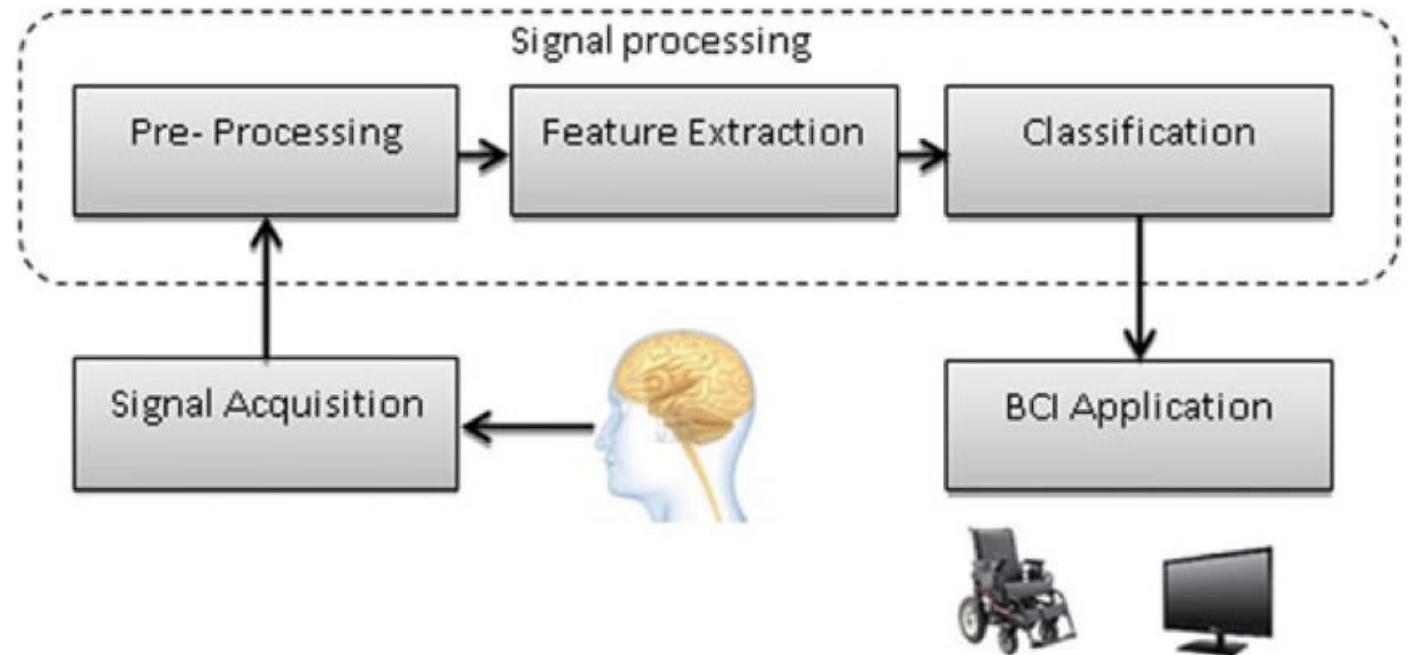
A BCI comprises:

- A **device** that records the brain activity (invasive/non-invasive)
- A **preprocessor** that reduces noise and artifacts, prepares the signals for further processing and extracts the relevant information from the recordings
- A **decoder** that classifies the extracted relevant information into a control signal for an **external device** that could be any type BCI-compatible application (e.g., a robotic actuator, a prosthesis, a computer screen etc.), and that provides feedback to the user



Design and implementation of a BCI

- **Preprocessing**
(artifact removal, filtering...)
- **Feature extraction in time or frequency domain**
(Frequency band power, autoregressive parameters, wavelet...)
- **Classification**
(threshold, support vector machine, neural networks)





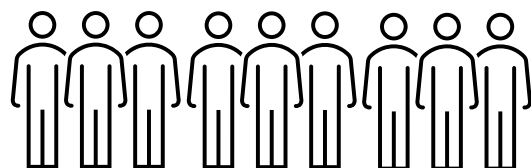
Preprocessing

- Data set description
- EEGLAB
- Filtering
- Re-reference
- Epochs extraction

Data set description: experimental paradigm

BCI Competition IV 2008, data set 2a

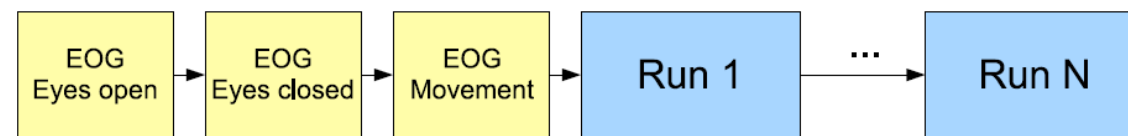
Subjects



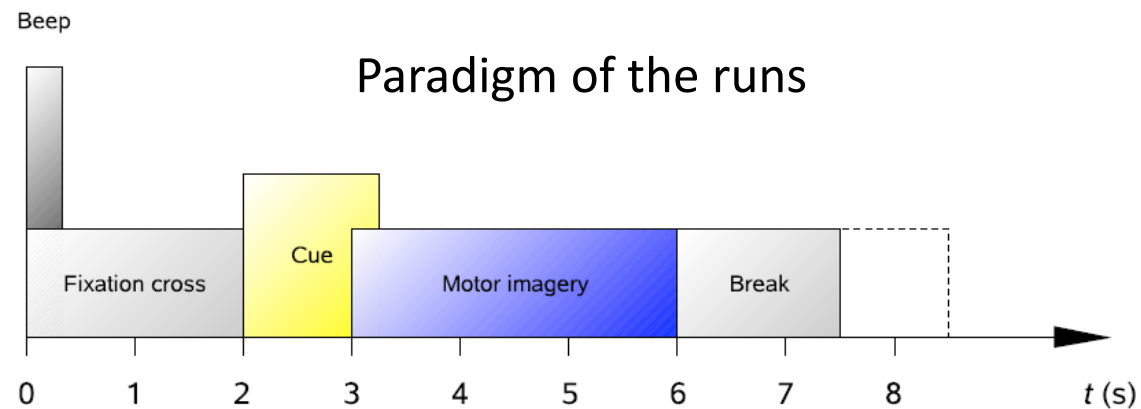
Motor imagery tasks



Recording of two sessions performed as follows:



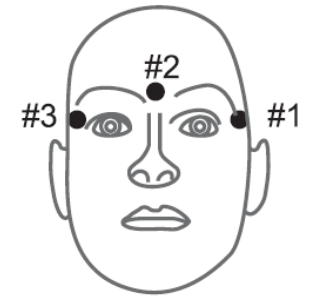
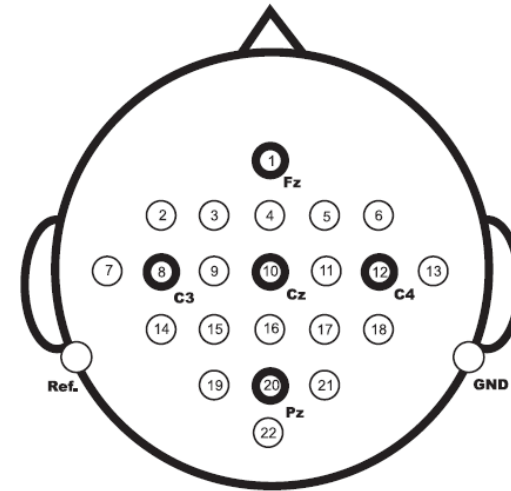
BCI Competition documentation



BCI Competition documentation

Data set description: data recording

- Signals **sampling**: 250 Hz + bandpass-filtering: 0.5 Hz - 100 Hz
- Notch filter to suppress line noise: 50 Hz
- Amplifier sensitivity: 100 μ V
- **EOG channels** for subsequent artifact processing
- Recorded data stored in **.gdf** (General Data Format) files



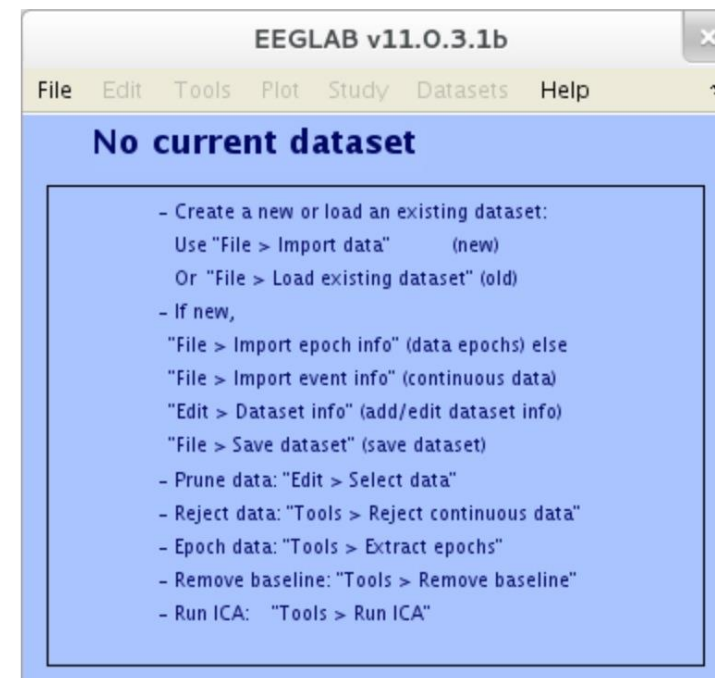
BCI Competition documentation

EEGLAB: initialization

```
% EEGLAB initialization  
[ALLEEG, EEG, CURRENTSET, ALLCOM] = eeglab;
```

- **ALLEEG:** array containing all the loaded EEG data sets
- **EEG:** current EEG data set
- **CURRENTSET:** index of the current data set
- **ALLCOM:** all the commands issued from the EEGLAB menu

Alternatively, the graphic user interface can be used by typing “**eeglab**” on the MATLAB command window and pressing enter.



EEGLAB: data set

```
% Loading the eeg data from file in a dedicated structure  
EEG = pop_biosig(filename);
```

EEG is a structure with some important fields like:

- **nbchan:** number of channels
- **srate:** sampling rate (250 Hz in this case)
- **times:** time at each sampled point (4 ms in this case, giving dimensions of the variable equal to 1x672528)
- **data:** EEG signal value for each time point (in this case, the dimensions of the variable are 25x672528)
- **event:** structure containing information about the recorded events
- **chanlocs:** structure containing information about the recording system

Some fields of the EEG structure must be filled with external information like, e.g., chanlocs (see the next slides).

EEGLAB: data set

Number of identification
depending on its description

Starting position in the data vector (where
each position represents a sample)

EEG.event						
Fields	type	edftype	edfplustype	latency	duration	urevent
1	'eeg:Idling EEG - eyes open'	276 []		1	29682	1
2	'boundary'	32766	'start of a new segment (after a break)'	29684	0	2
3	'eeg:Idling EEG - eyes closed'	277 []		29684	20271	3
4	'boundary'	32766	'start of a new segment (after a break)'	49956	0	4
5	'1072'	[] []		49956	41562	5
6	'boundary'	32766	'start of a new segment (after a break)'	91519	0	6
7	'Start of Trial, Trigger at t=0s'	768 []		91869	1875	7
8	'class4, Tongue- cue onset (BCI experiment)'	772 []		92369	313	8
9	'Start of Trial, Trigger at t=0s'	768 []		93872	1875	9
10	'class3, Foot, towards Right - cue onset (BCI experiment)'	771 []		94372	313	10
11	'Start of Trial, Trigger at t=0s'	768 []		95790	1875	11
12	'class2, Right hand- cue onset (BCI experiment)'	770 []		96290	313	12

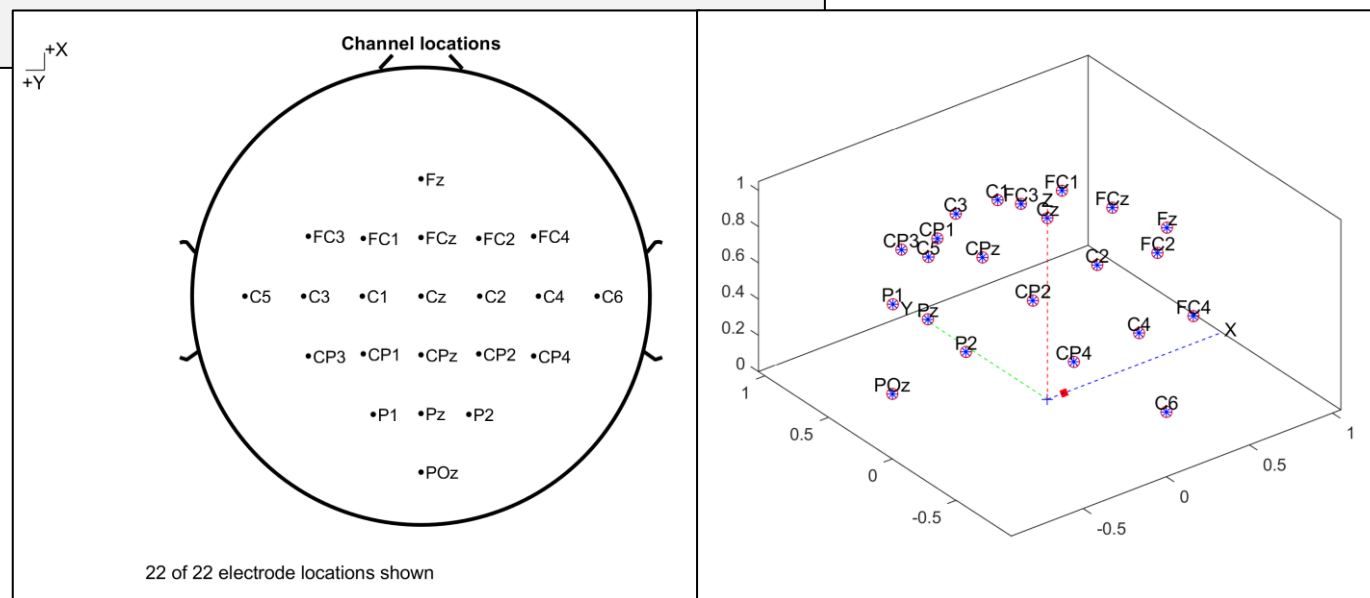
Description

Number of samples along which the
event in object has been recorded

EEGLAB: data set

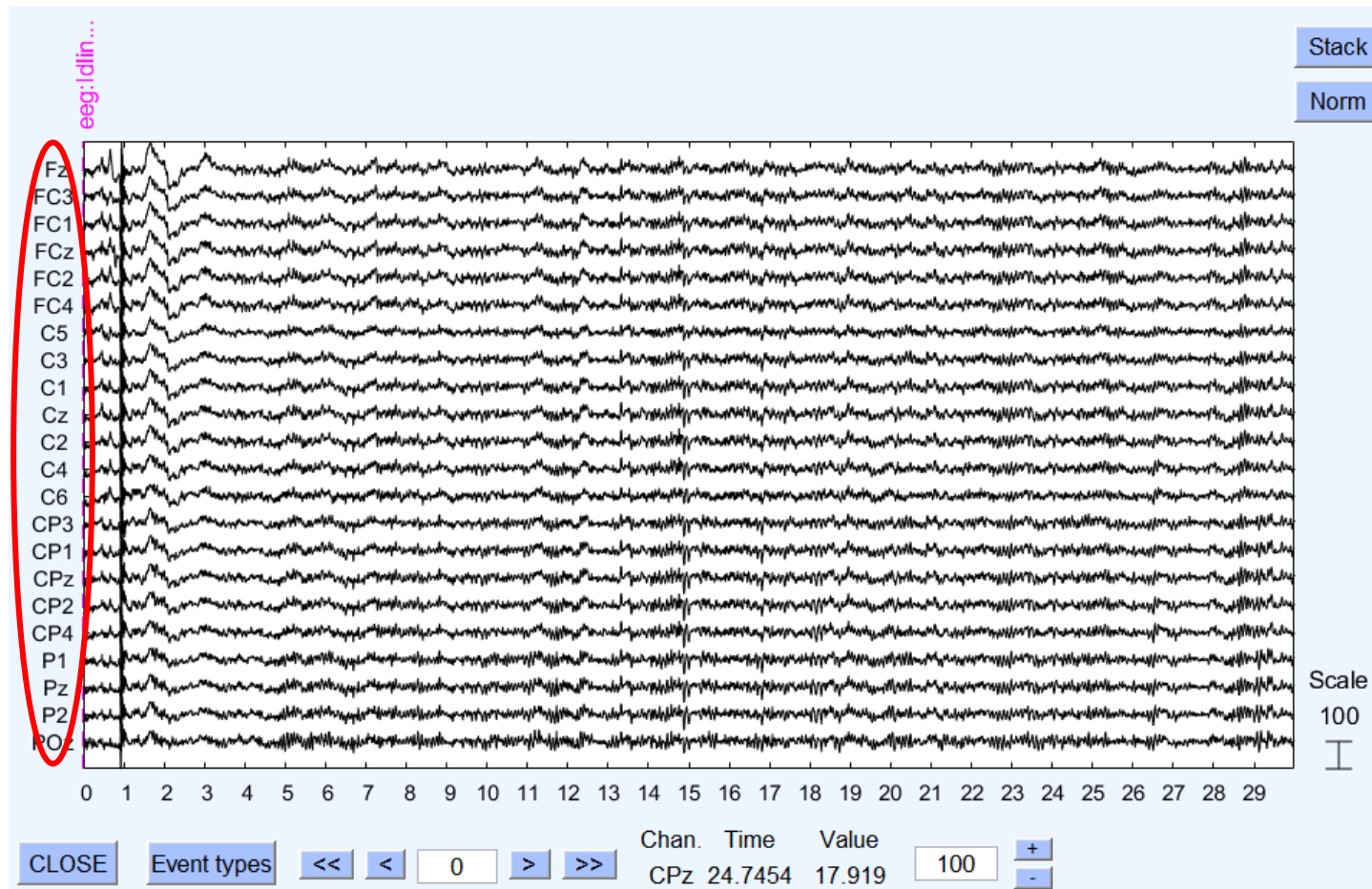
```
% Discarding the channels corresponding to the three monopolar EOG  
% channels  
EEG = pop_select(EEG, 'nochannel', [23, 24, 25]);  
  
% Update the field dedicated to channels using the file .locs  
chanlocs22 = readlocs('chanlocs22.locs');  
EEG.chanlocs = chanlocs22;
```

Defined by a **label** and a **position**
(in both Cartesian and spherical
coordinates) for each channel



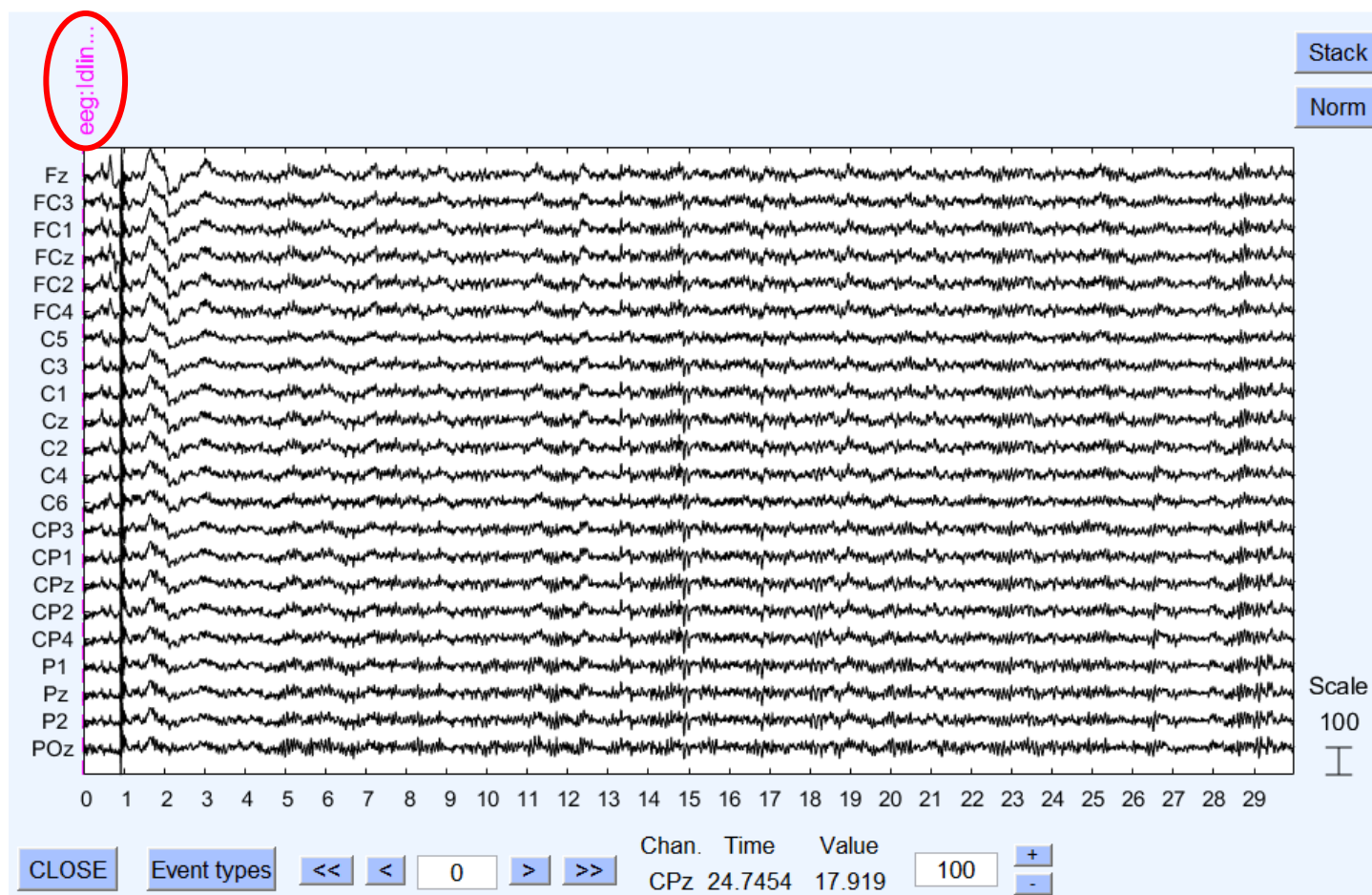
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



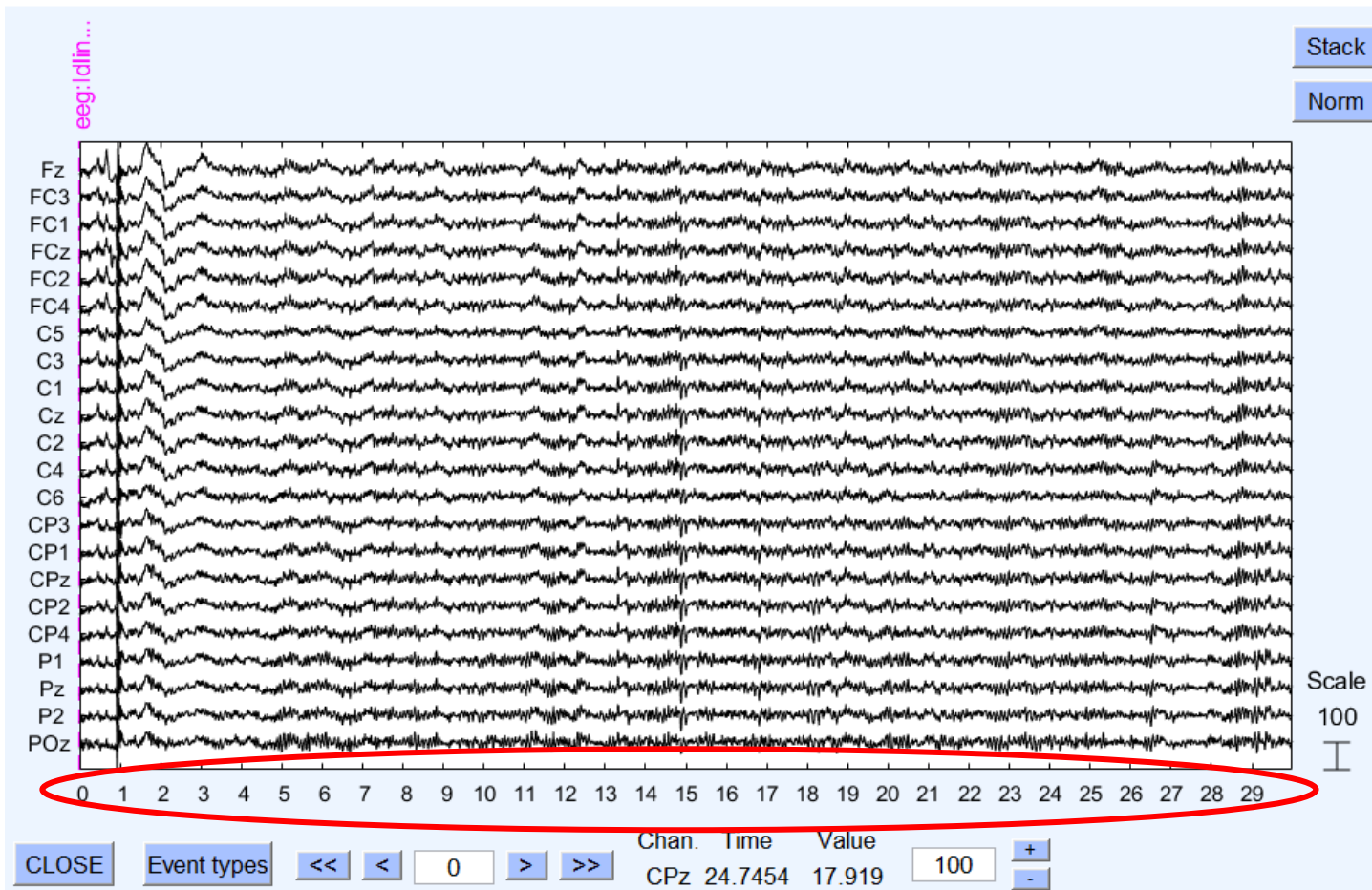
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



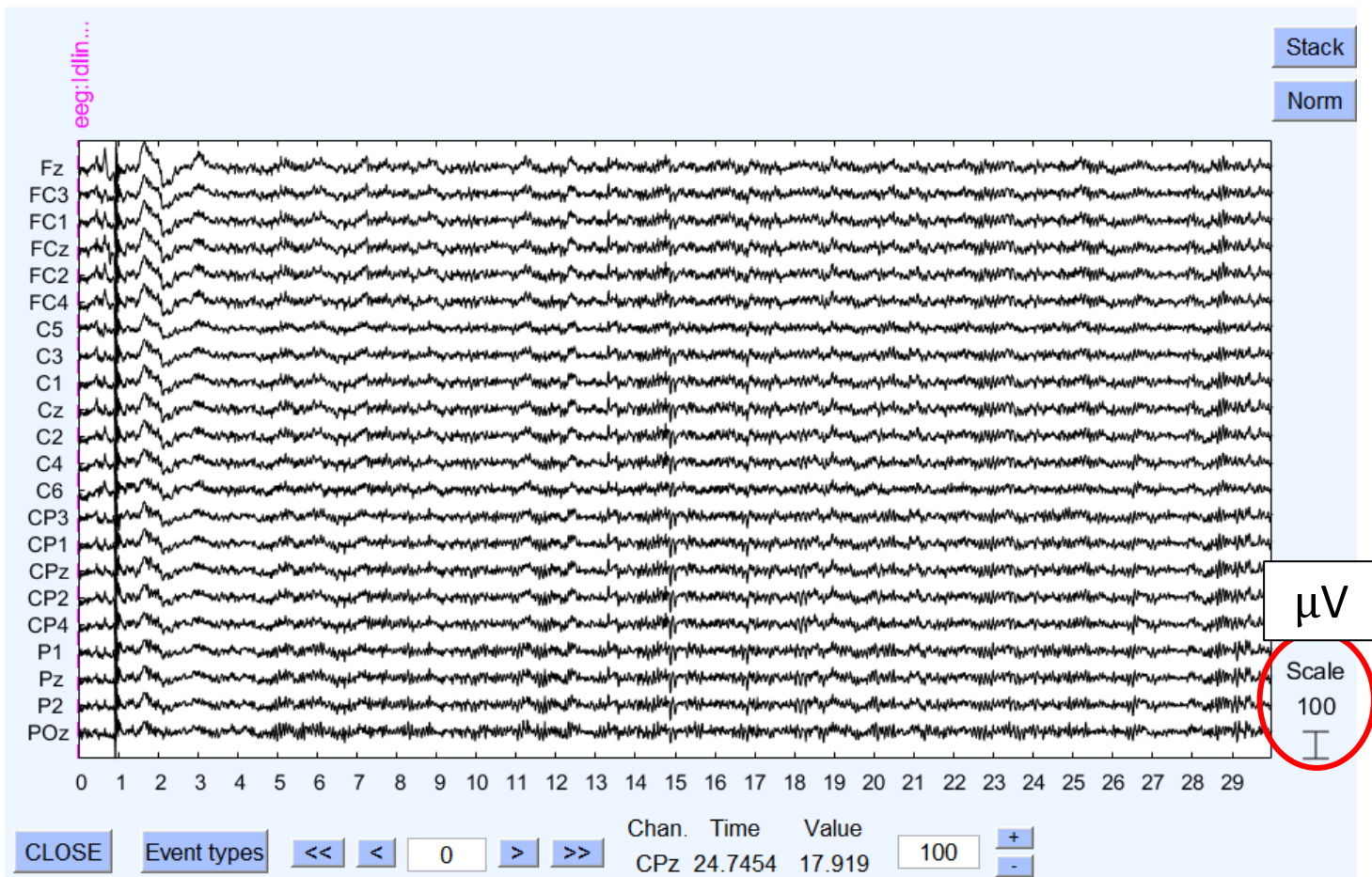
EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



EEGLAB: data set

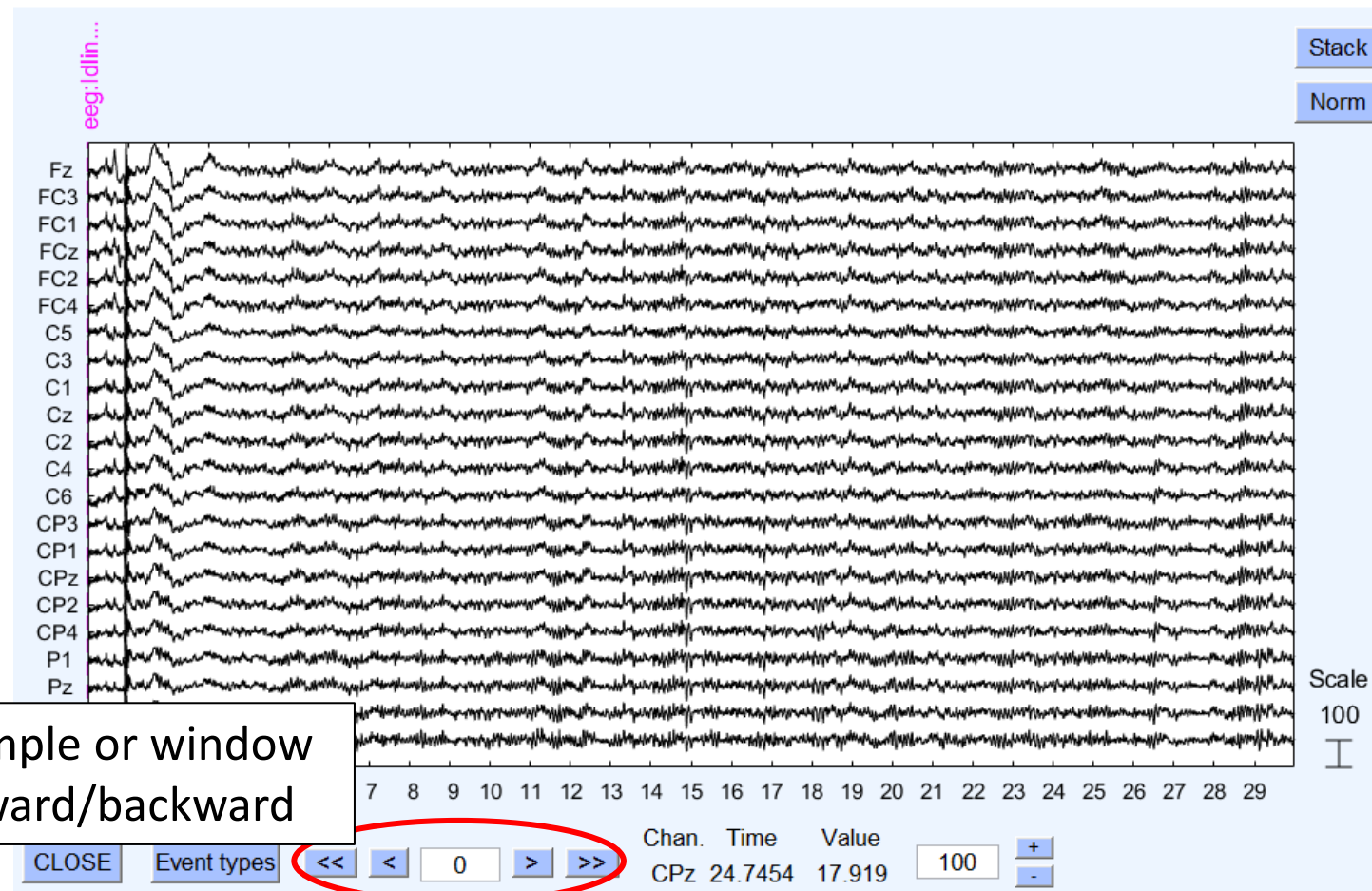
```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```



EEGLAB: data set

```
% Displaying the raw eeg signal
eegplot(EEG.data,...
    'title', 'Raw EEG',...
    'srate', EEG.srate,...
    'eloc_file', EEG.chanlocs,...
    'events', EEG.event,...
    'winlength', winlength,...
    'color', color,...
    'spacing', 100);
```

1 sample or window
forward/backward



EEGLAB: useful commands

```
% Storing the current EEG data  
[ALLEEG, EEG, CURRENTSET] = eeg_store(ALLEEG, EEG);
```

- **ALLEEG:** now contains, in addition, the data set stored in EEG (the current one)
- **CURRENTSET:** index to the current set

```
% Creating a new EEG copy  
[ALLEEG, EEG, CURRENTSET] = pop_newset(ALLEEG, EEG, CURRENTSET, 'setname',  
'EEG_BAS');
```

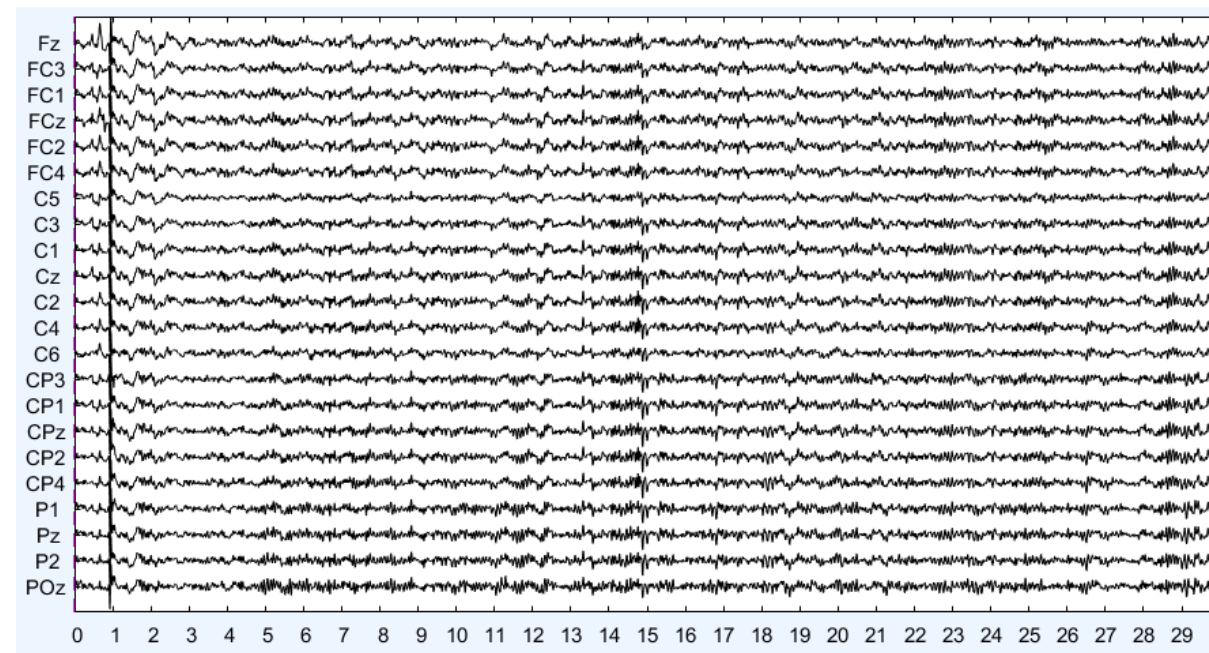
- **EEG:** the EEG structure manipulated until now is renamed as 'EEG_BAS'
- **ALLEEG:** now contains, in addition, the data set 'EEG_BAS'
- **CURRENTSET:** the current set, at this point, could be the second one in ALLEEG

Filtering

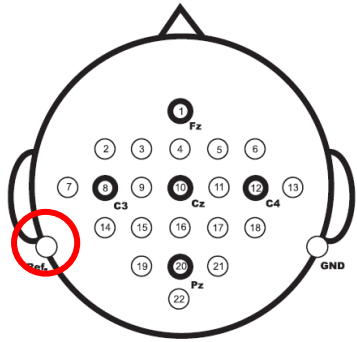
1. Baseline removal
 - Do the mean of the signal for each channel
 - For each signal value, subtract the corresponding mean for all the channels
2. Retain frequencies in band 1-30 Hz

In EEGLAB is advised to do:

```
% High pass filter
EEG = pop_eegfilt(EEG, 1, 0);
% Low pass filter
EEG = pop_eegfilt(EEG, 0, 30);
```



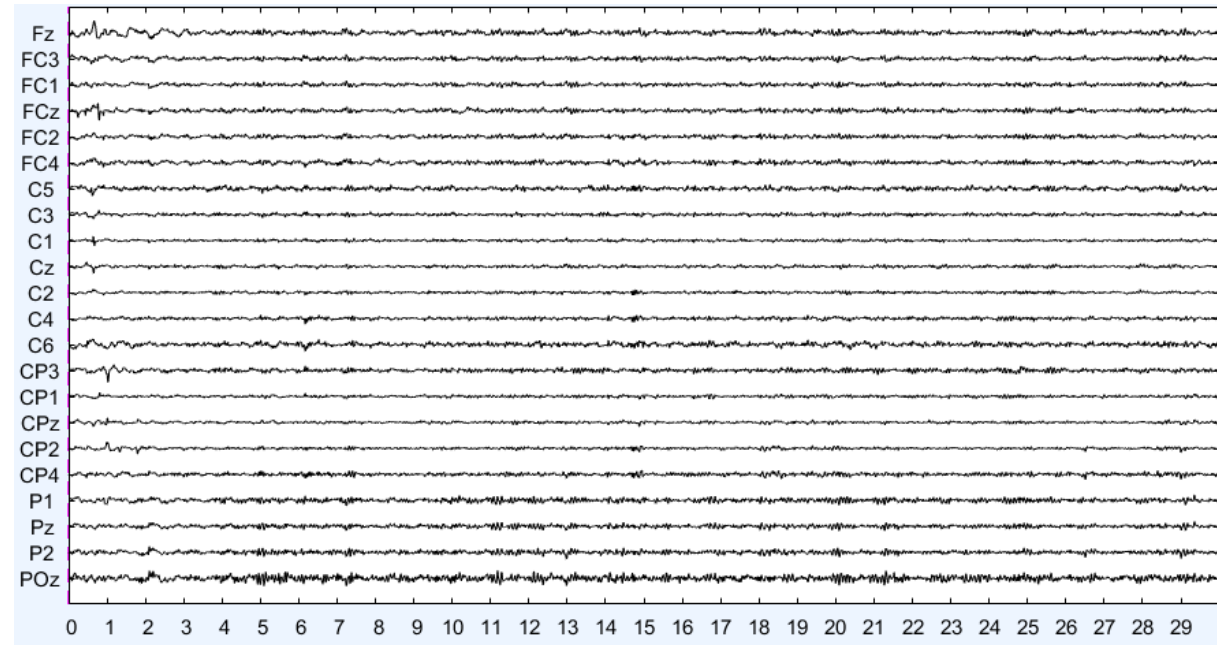
Re-reference



BCI Competition
documentation

Classical EEG signal data of each channel are recorded as a difference between the electrical potential in the channel location and a **reference**.

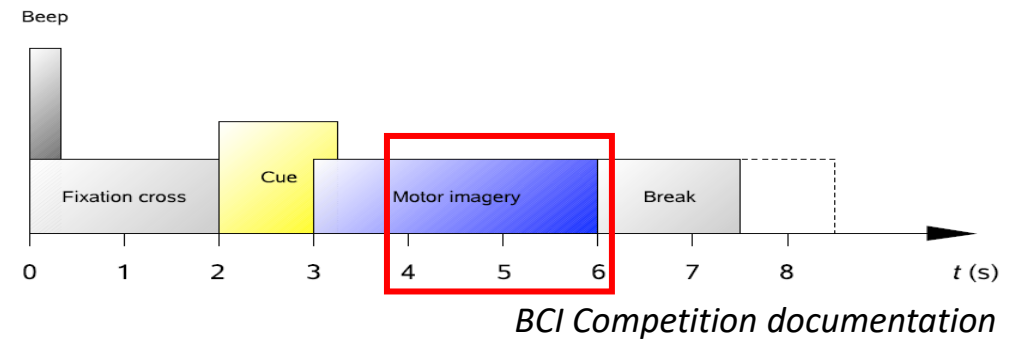
```
% Re-referencing the data with  
respect to average reference  
EEG = pop_reref(EEG, []);
```



Epochs extraction

The interesting events are those corresponding to motor imagery tasks:

- Left Hand: `EEG.event(i).edftype = 769`
- Right Hand: `EEG.event(i).edftype = 770`
- Both Feet: `EEG.event(i).edftype = 771`
- Tongue: `EEG.event(i).edftype = 772`



```
switch EEG.event(i).edftype
case 769
    EEG.event(i).type = "Left_Hand";
    EEG.event(i).latency = EEG.event(i).latency + sampling*2;
    EEG.event(i).duration = sampling*2;
[...]
% EEG.data(:, EEG.event(i).latency:(EEG.event(i).latency + EEG.event(i).dur
% ation - 1)));
```

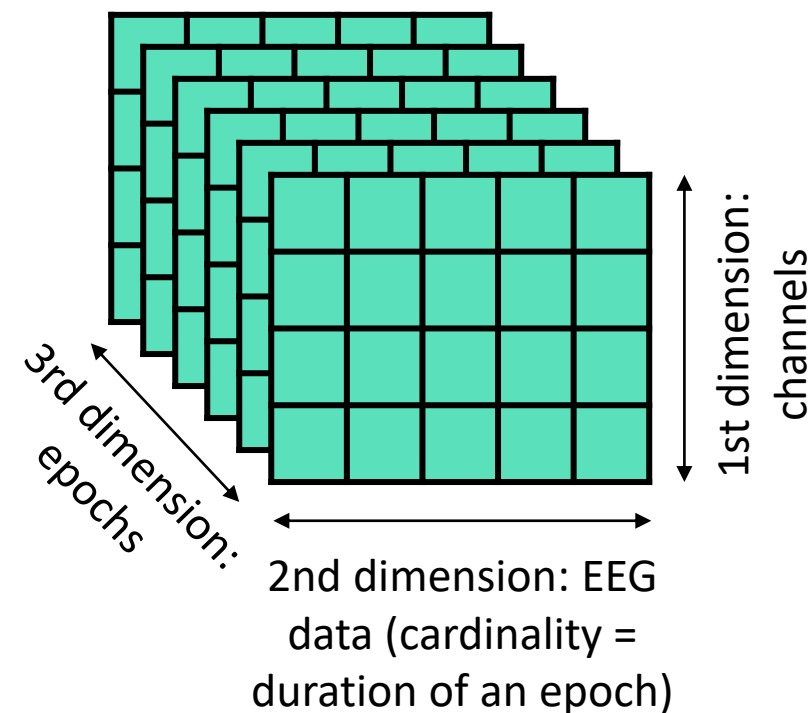
Epochs extraction

Let's save the events of interest in a EEG field called epoch...

And focus, e.g., on the event "Right Hand"

```
% Access to epochs belonging to the  
% event of interest  
EEG.epoch(3).data(:, :, 1);
```

EEG.epoch			
Fields	str	type	data
1		"Tongue"	22x500x72 single
2		"Both_Feet"	22x500x72 single
3		"Right_Han..."	22x500x72 single
4		"Left_Hand"	22x500x72 single



Exercise

Brain Computer Interfaces (2020/2021)

[Dashboard](#) / [My courses](#) / [Brain Computer Interfaces \(2020/2021\)](#) / [Laboratory](#) / [Hands-On](#)

Hands-On

- ▼ 
 - ▼  BCICompetitionIVDataset2A
 - ▼  ClassificationResults
 - ▼  Training
 - ▼  Features
 - ▼  Training
 - ▼  Preprocessed
 - ▼  Training
 - ▼  Samples
 - ▼  Training
 - ▼  A01T.gdf
 - ▼  01_preprocessing_exercise.m
 - ▼  chanlocs22.locs

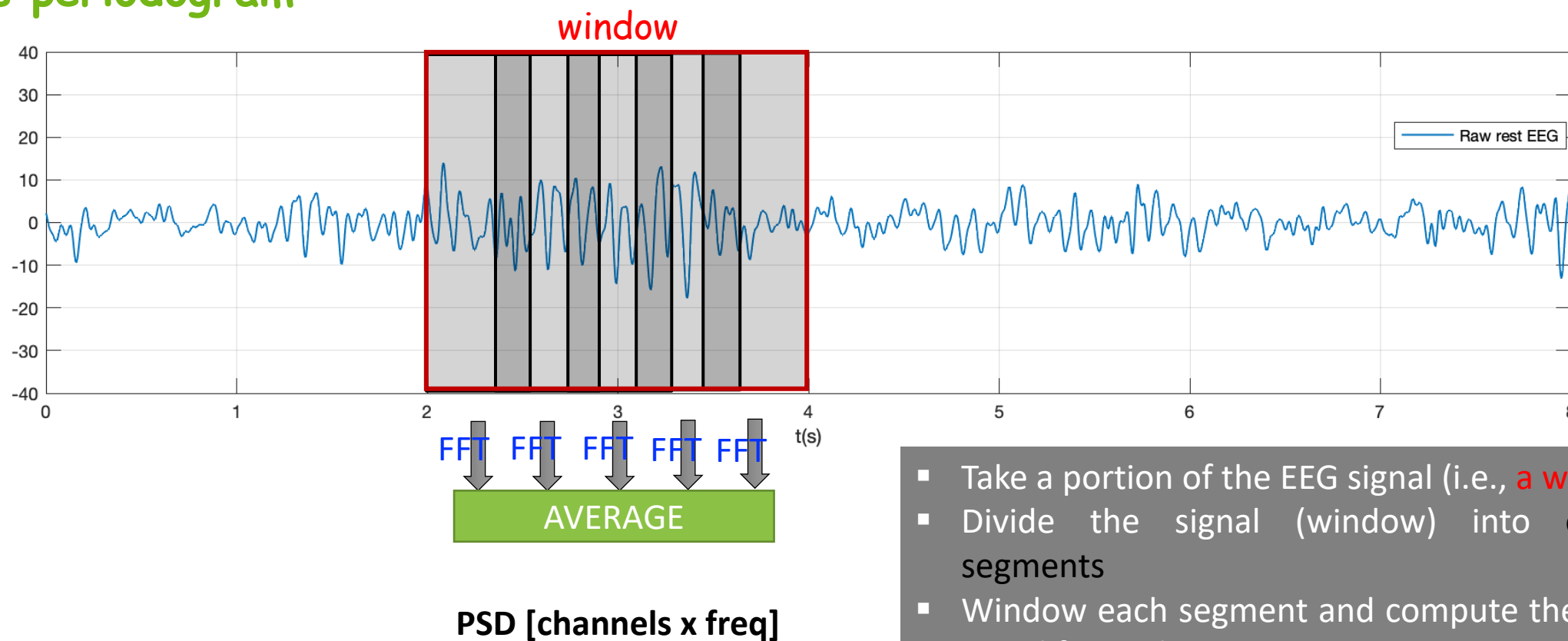


Feature extraction

- Power Spectral Density
- Coherence
- Correlation
- Feature storing

Power Spectral Density

Welch's periodogram



- Take a portion of the EEG signal (i.e., a **window**)
- Divide the signal (window) into overlapping segments
- Window each segment and compute the FFT of the signal for each segment
- Average the periodograms of the segment

Power Spectral Density

EEGLAB provides the function **spectopo**:

```
[spectra,freqs] = spectopo(data,  
frames, srate, 'overlap');
```

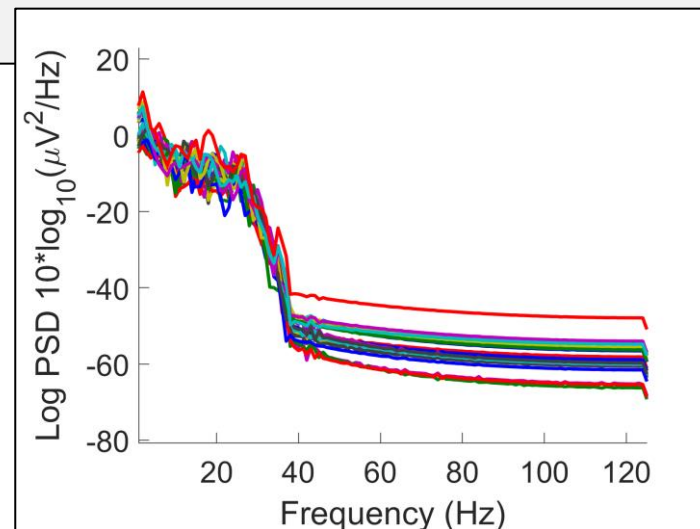
Input

- **data**: it can be a 2-D matrix corresponding to a single epoch
- **frames**: number of samples in an epoch
- **srate**: sampling rate
- **'overlap'**: desired overlapped samples

Output

- **spectra**: power spectra in dB (nchans x nfreqs)
- **freqs**: frequencies of the spectra in Hz

```
% Calculate the power spectral density for  
% all the epochs  
for i = 1:nepochs  
    [spectra_RH(:, :, i), freqs_RH(:, :, i)] =  
        spectopo(EEG.epoch(3).data(:, :, i),  
length(EEG.epoch(3).data(:, :, i)),  
EEG.srate, 'overlap', EEG.srate/2);  
end
```



Power Spectral Density

The EEGLAB function `spectopo`, internally calls the MATLAB function **`pwelch`**:

```
[spectra,freqs] = pwelch(data,  
window, overlap, nfft, srate);
```

Input

- **window:** if it is an integer, `pwelch` is divided into segments of length equal to the integer value
- **nfft:** number of DFT points

```
for i = 1:nepochs  
    [spectra_tmp,freqs_RH(:, :, i)] =  
        pwelch(EEG.epoch(3).data(:, :, i)',  
        EEG.srate, EEG.srate/2, EEG.srate,  
        EEG.srate);  
    spectra_RH(:, :, i) = 10*log10(spectra_tmp');  
end
```

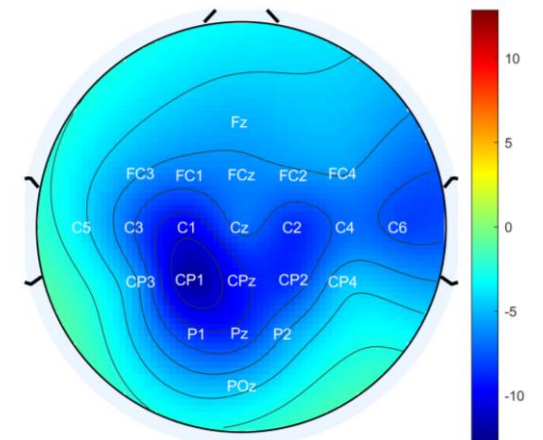

Power Spectral Density

It is usual to calculate the **mean** PSD over specific **band** frequencies:

```
% Retrieve indices for alfa frequencies
idx_alfa = (freqs_RH(:,1,1) >= 8 & freqs_RH(:,1,1) < 13);
% Calculate the mean PSD over alfa band
psd_RH_alfa = mean(spectra_RH(:,idx_alfa,:),2);
```

- Now, psd_RH_alfa has dimensions: nchans x 1 x nepochs

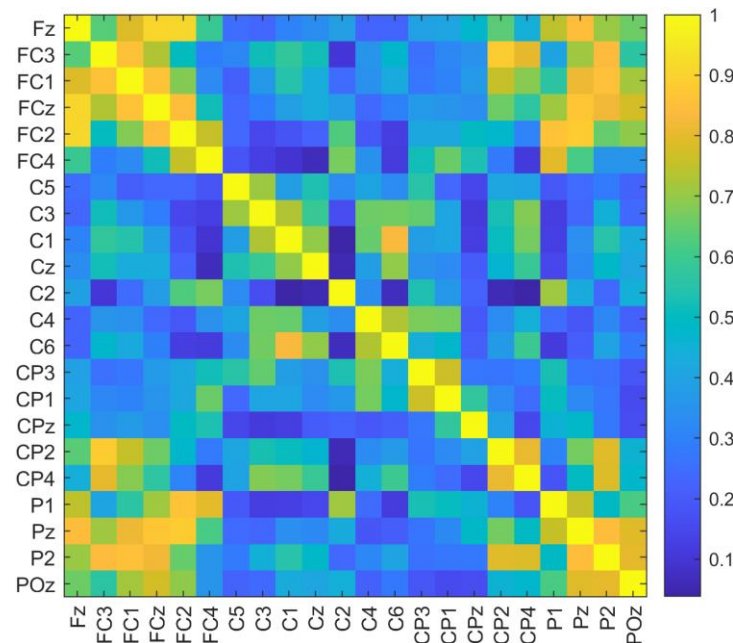
```
% Visualize a topographic scalp map of the "Right Hand"
% PSD alfa in the first epoch
topoplot(psd_RH_alfa(:,1,1), 'chanlocs22.locs',
         'electrodes', 'labels', 'ecolor', 'white');
```



Coherence

COHERENCE (or MAGNITUDE-SQUARE COHERENCE)

is a measure of degree of association or coupling of frequency spectra between different times series. The **coherence** is intended as the ratio between the cross-spectrum modulus and the product of the two autospectra (the coherence is basically a normalized cross spectrum).



Coherence

MATLAB provides the function `mscohere`

```
[coh, freqs] = mscohere(x, data, window, overlap, nfft, srate);
```

- **x**: vector containing the signal coming from one channel
- **coh**: magnitude-squared coherence estimate; according to the presented input, it has dimensions `freqs x nchans`

```
coh_alfa_RH = ones([EEG.nbchan, EEG.nbchan, nepochs]);  
for i = 1:size(EEG.epoch(3).data, 3)  
    for j = 1:EEG.nbchan  
        channel = EEG.epoch(3).data(j, :, i)';  
        data = EEG.epoch(3).data(:, :, i)';  
        [coh, f] = mscohere(channel, data, EEG.srate, EEG.srate/2, EEG.srate, EEG.srate);  
        coh_alfa_RH(j, :, i) = mean(coh(idx_alfa, :), 1);  
    end  
end
```

Correlation

CORRELATION

Let s_x and s_y denote, respectively, the standard deviations and cov_{xy} the covariance of two time series x and y , the functional connectivity simply corresponds to the **Pearson's correlation coefficient r** of x and y

$$r_{xy} = \frac{\text{cov}_{xy}}{s_x s_y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

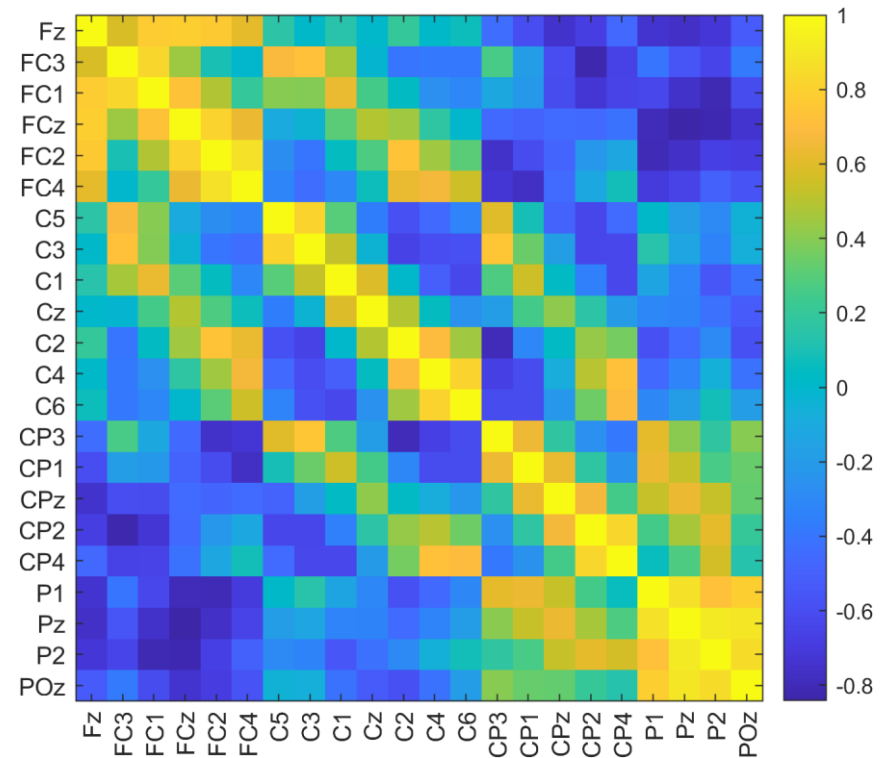
when $r > 0$ we say that the time series are **positively correlated**, and when $r < 0$ we say that they are **negatively correlated**.

Correlation

MATLAB provides the function `corrcoef` for the calculation of the Pearson's correlation coefficient.

To visualize a brain connectivity matrix:

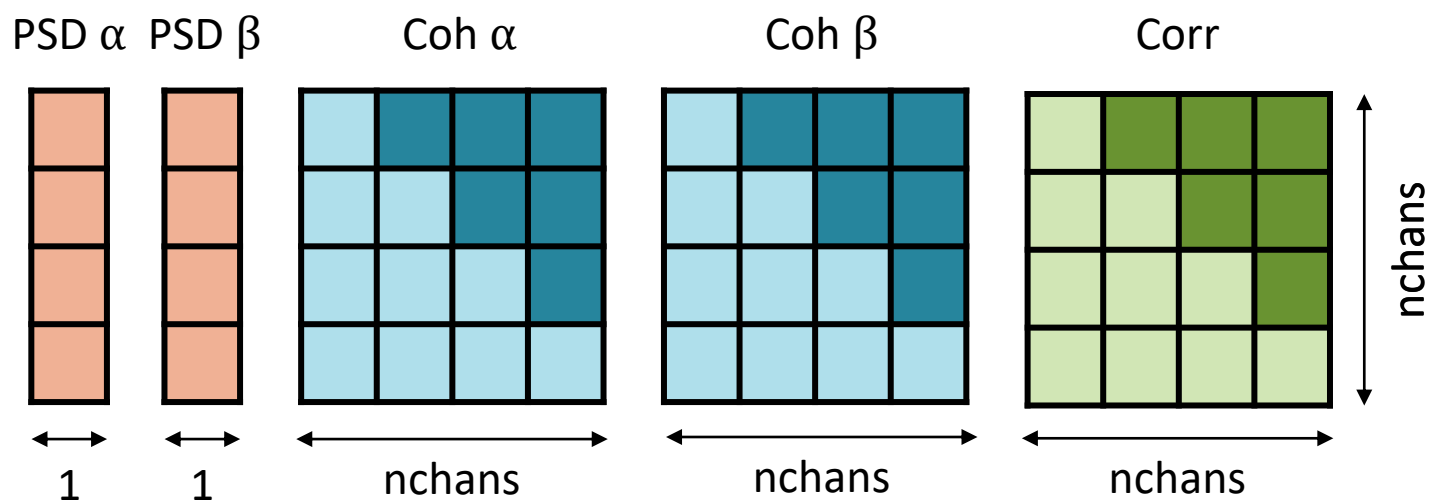
```
figure;  
imagesc(coh_alfa_RH(:, :, 1));  
axis square;  
colorbar;  
xticks(1:1:EEG.nbchan);  
yticks(1:1: EEG.nbchan);  
xticklabels(chanlabels);  
yticklabels(chanlabels);  
xtickangle(90);
```



Feature storing

Summary of the seen features and their dimensionality.

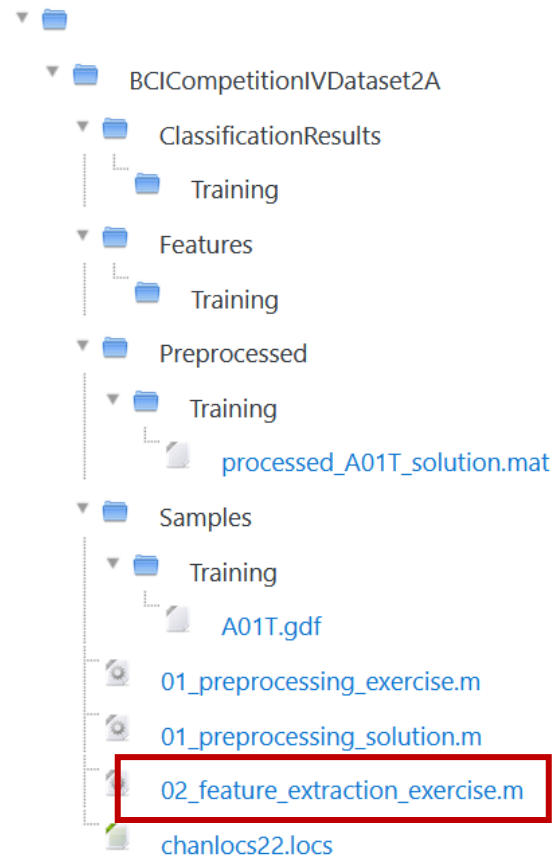
For each event (“Right Hand” or “Both Feet”) and for each epoch (in this case, 72):



```
% Transform features in row  
% vectors, one for each epoch.  
% For PSD  
PSD_RH_alfa =  
psd_RH_alfa(:, :, 1)';  
% For connectivity  
mask =  
ones([EEG.nbchan, EEG.nbchan]);  
mask = triu(mask, 1);  
COH_RH_alfa = coh_RH_alfa(:, :, 1);  
COH_RH_alfa =  
COH_RH_alfa(logical(mask));
```

Exercise

Hands-On

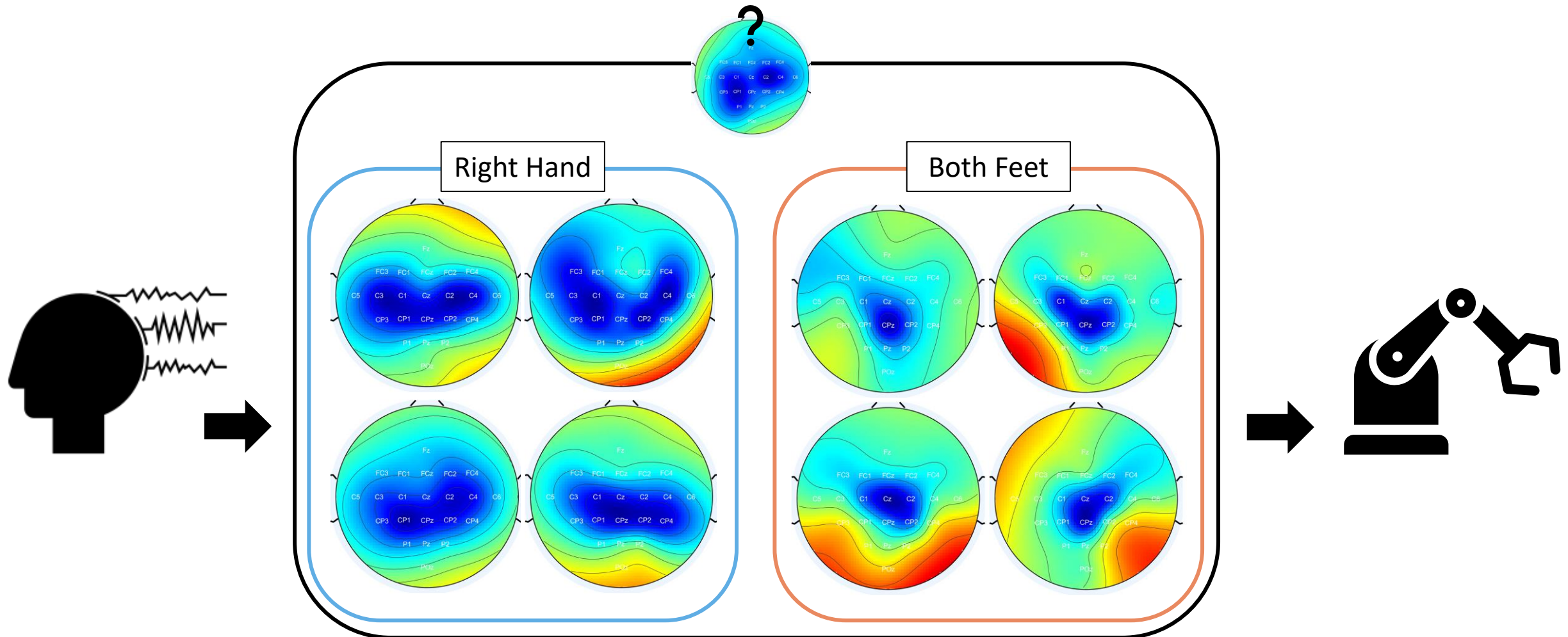




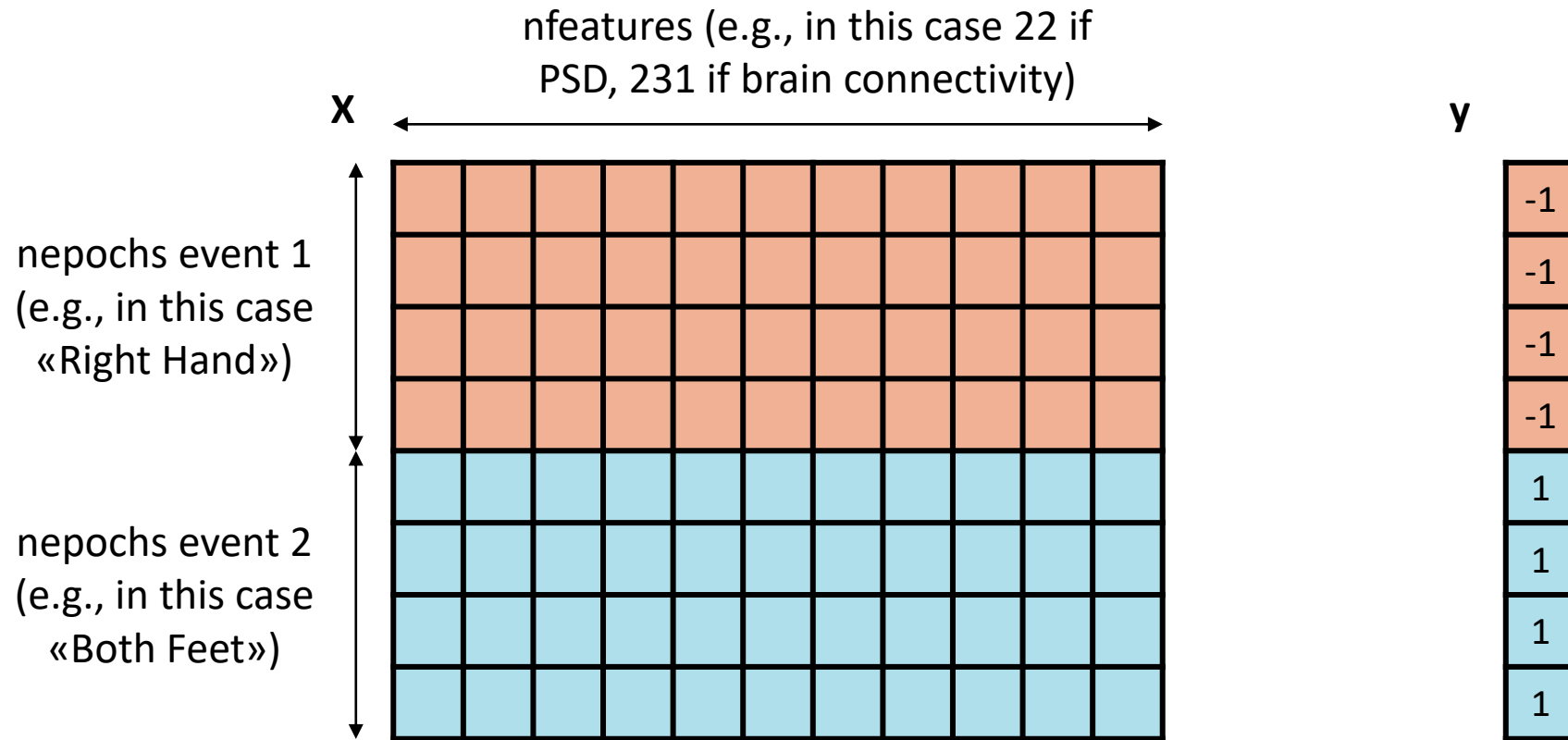
Classification

- The classification in BCI
- Machine Learning - a means for BCI
- Evaluating the training
- Classification performance measurements
- Testing
- BCI classifiers
- Linear Discriminant Analysis Classification
- Support Vector Machine

The classification in BCI



The classification in BCI



Machine Learning - a means for BCI

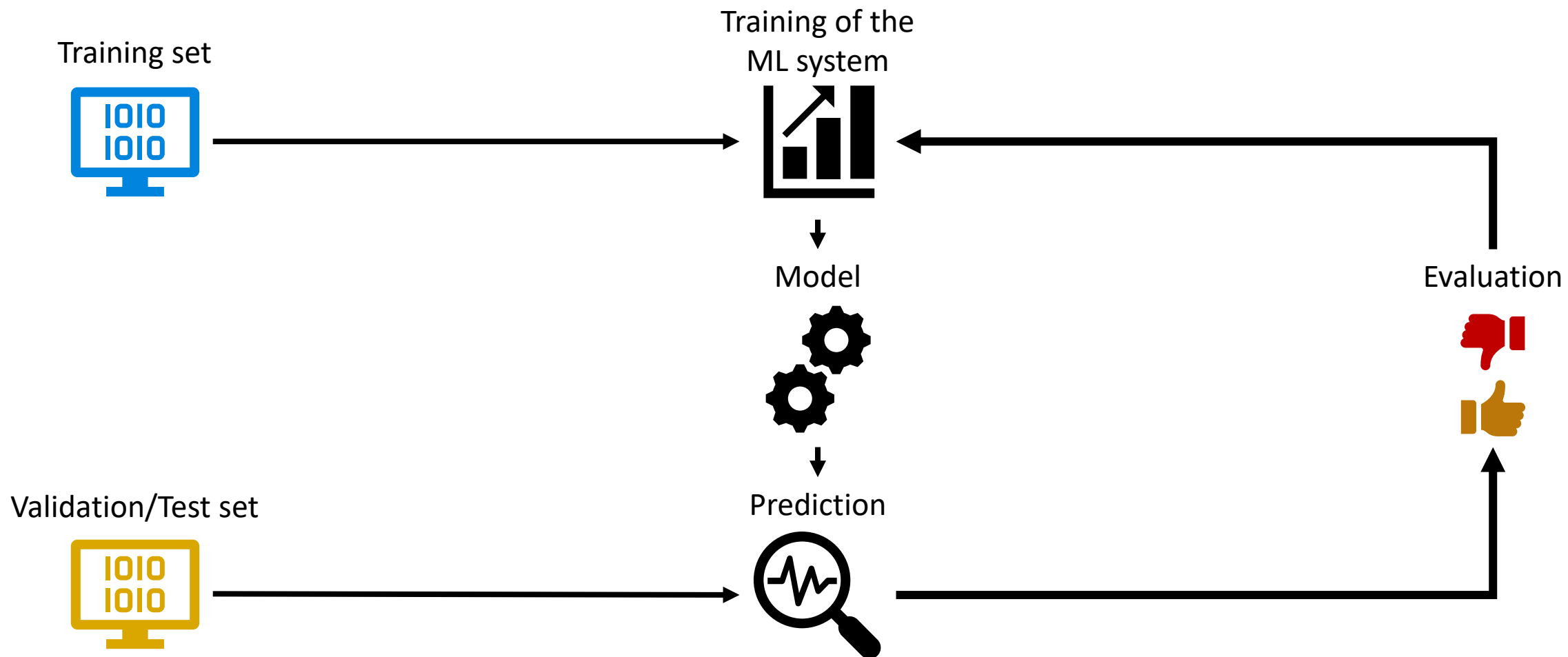
[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

- Arthur Samuel, 1959

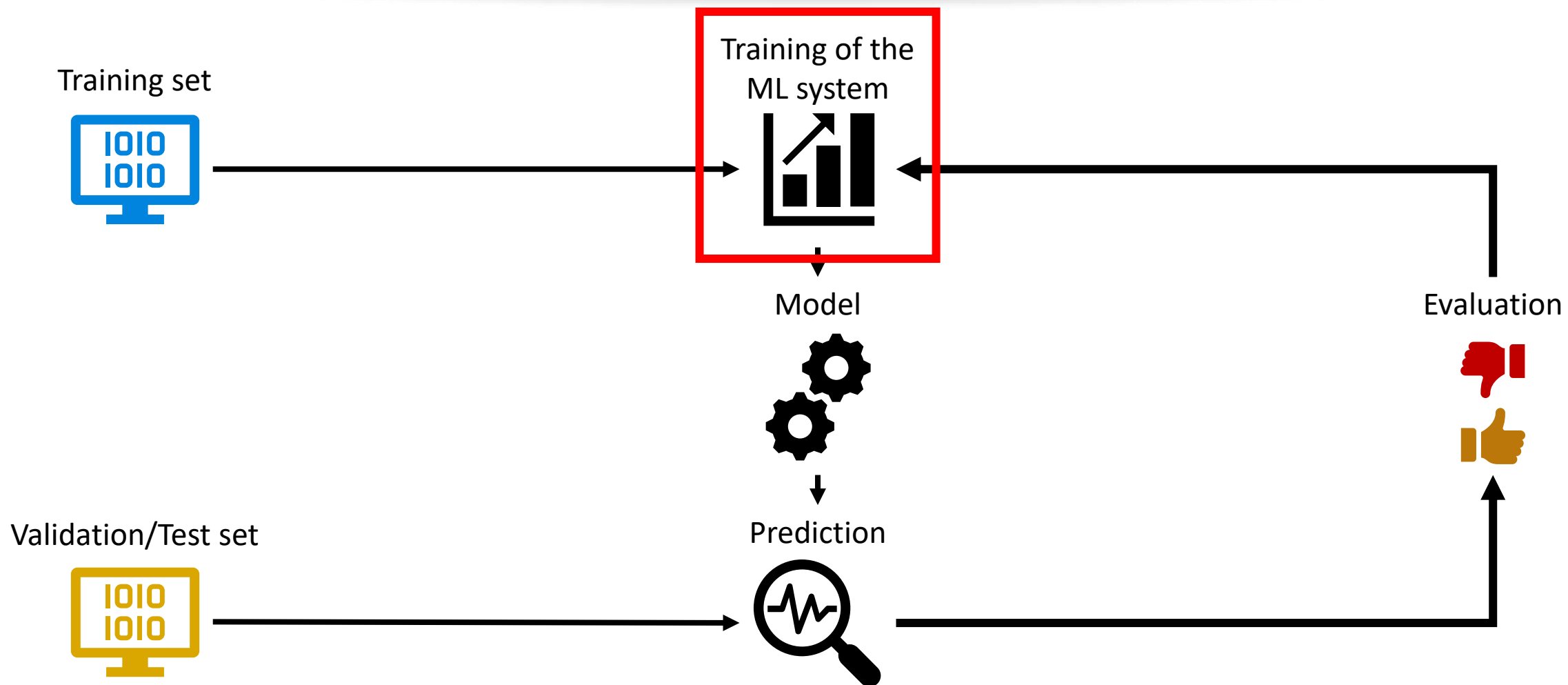
Machine learning systems can be:

- **Supervised:** learning from a data set including the desired solutions (labels)
- **Unsupervised:** learning from an unlabeled data set (i.e., without a teacher)
- **Semisupervised:** learning from a partially labeled data set
- Based on **reinforcement:** learning by i) observing the environment, ii) selecting and performing actions using a policy, iii) using the obtained reward or penalty to update the policy, iv) iterating the policy until the reaching of the optimal one

Machine Learning - a means for BCI



Machine Learning - a means for BCI

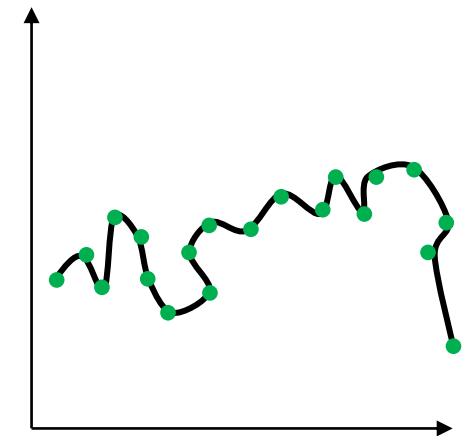


Evaluating the training: challenges

Overfitting: the model performs well on the training data, but it does not effectively generalize

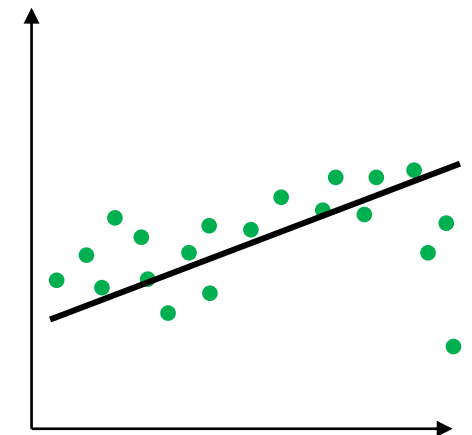
Possible solutions:

- Simplify the model
- More training samples
- Reduce the noise



Underfitting: the model is too simple to learn the underlying structure of the data

- Feature engineering
- Reduce the constraints on the model



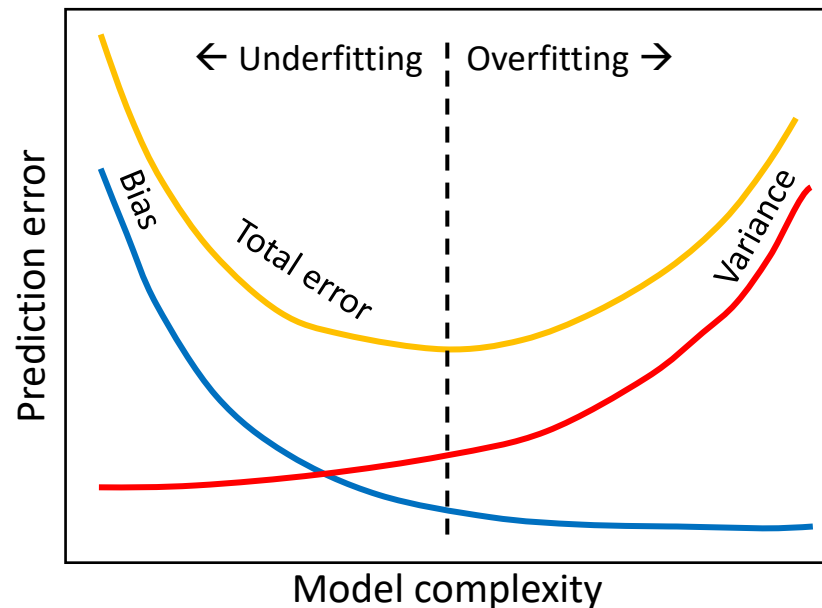
Evaluating the training: tradeoff

Bias

Error due to the assumptions of the model used to fit the data. When the bias is high, there is the possibility to **underfitting** the system – that is the missing of relevant relations between dependent and independent variables.

Variance

Error due to the sensitivity to small fluctuations in the data. When the variance is high, there is the possibility to **overfitting** the system – that is the low capability to generalize.



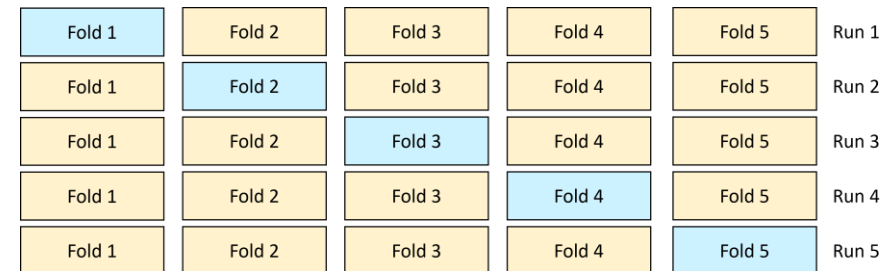
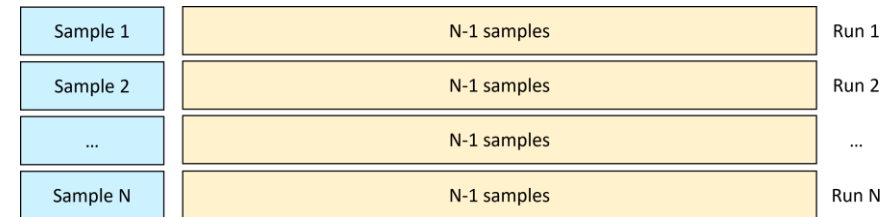
Tradeoff

In machine learning, the ideal algorithm has low bias and low variability which can be obtained by looking for a compromise between a simple model and a complex one.

Evaluating the training: testing and validating

Since it is desirable to use the **test** set only when we are confident about the model, it is possible to evaluate the **training** performance by using part of the training set for fitting and part for **validation**.

- **HOLDOUT:** randomly split the training set in two parts in a fixed proportion (e.g., 80-20%); iterate for a high number of times
- **LEAVE-ONE-OUT:** use all the samples for training except one; iterate over all the samples
- **CROSS-FOLDING:** split the training set into K folds; use K-1 folds for training and validate on the remaining one; iterate over all the folds



Evaluating the training: testing and validating

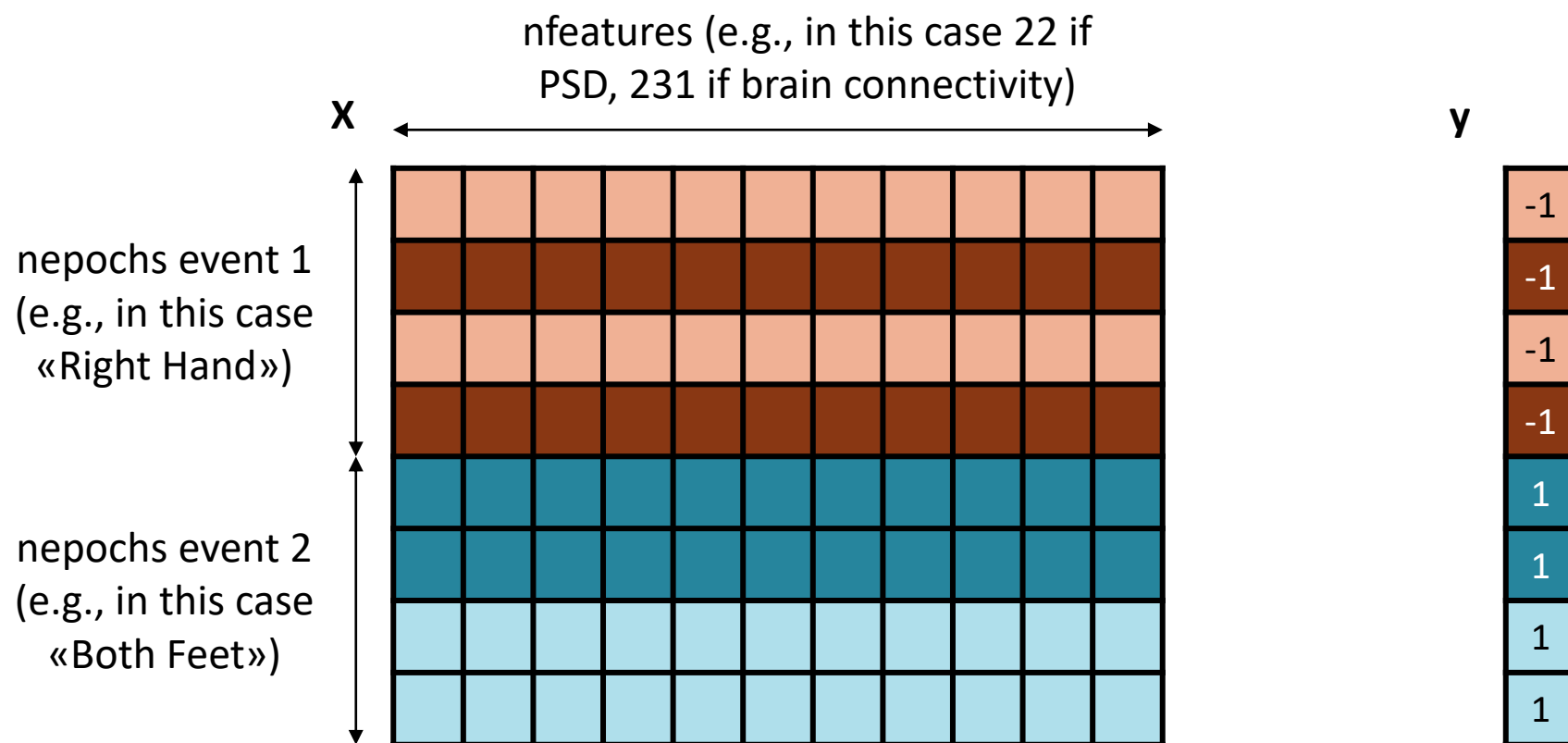
The functions **cvpartition**, **training** and **test** are used to divide the data set in the desired partitions.

```
% Create a holdout partition object
c = cvpartition(y, 'Holdout', 0.2, 'Stratify', true);
% Extract the indices for row selection
idx_training = training(c); idx_test = test(c);

% Create a k-fold partition object
c_kfold = cvpartition(y, 'KFold', 5, 'Stratify', true);
```

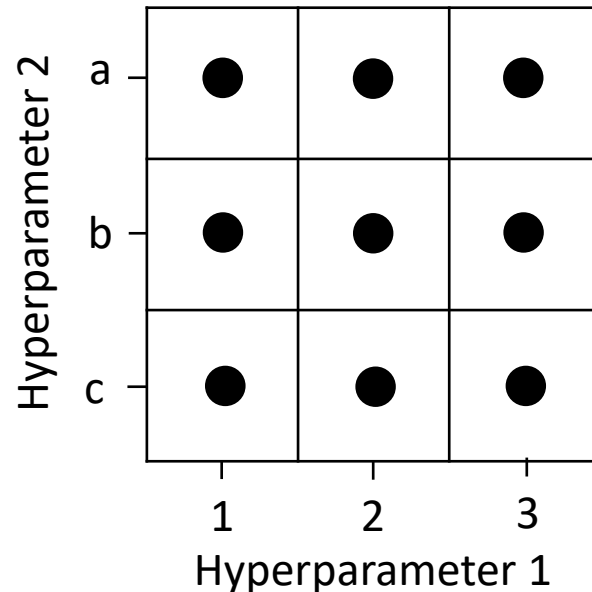
- **y**: vector containing all the labels, one for each sample
- **'Holdout'** or **'KFold'**: depending on the desired type of partition, the following input specifies the percentage of samples for test on the total for 'Holdout', and the number of folds for 'KFold'
- **'Stratify'**: if true, the partition is created by equalizing the percentage of the classes across partitions

Evaluating the training: testing and validating



Evaluating the training: regularization

- Constraining a model to make it simpler is called **regularization**
- The amount of regularization to apply during learning can be controlled by a **hyperparameter**
- The hyperparameter is a parameter of the **learning algorithm** (not of the model!)
- The hyperparameter must be set **prior** to training and remains constant during the training
- Repeated cross-validation allows to perform the hyperparameter **tuning**



Evaluating the training: regularization

MATLAB alternatives for doing model regularization:

- **Classification Learner app**
- **fitcauto** function (it employs Bayesian optimization)
- **ad hoc** hyperparameters tuning of a specific model

```
% Create a structure with options for optimization
opts = struct('Optimizer', 'gridsearch', 'NumGridDivisions', 20,...
    'ShowPlots', false, 'CVPartition', c_kfold);
% Fit a SVM with optimization of the hyperparameters as specified by opts
svm_optimization = fitcsvm(X_training, y_training,...
    'KernelFunction', 'linear', 'Standardize', true,...
    'OptimizeHyperparameters', 'auto',...
    'HyperparameterOptimizationOptions', opts);
```

Evaluating the training: regularization

Main options for the optimization:

- **'Optimizer'**: it can be
 - *'bayesopt'* – use Bayesian optimization
 - *'gridsearch'* – perform a search by examining all combinations of the hyperparameters (i.e., grid) selected by uniformly sampling their values. The 'NumGridDivision' defines the number of values in each dimension
 - *'randomsearch'* – search at random
- **'ShowPlots'**: if true, plot the best objective function value against the iteration number
- **'CVPartition'**: use a cross-validation to evaluate the performance

```
Optimization completed.  
MaxObjectiveEvaluations of 400 reached.  
Total function evaluations: 400  
Total elapsed time: 174.6497 seconds  
Total objective function evaluation time: 168.1856
```

Best observed feasible point:

BoxConstraint	KernelScale
0.69519	6.1585

```
Observed objective function value = 0.12931  
Function evaluation time = 0.04706
```

Evaluating the training: dimensionality reduction

Curse of dimensionality: the more dimensions the training set has, the greater the risk of overfitting it!

Dimensionality reduction can be a solution.

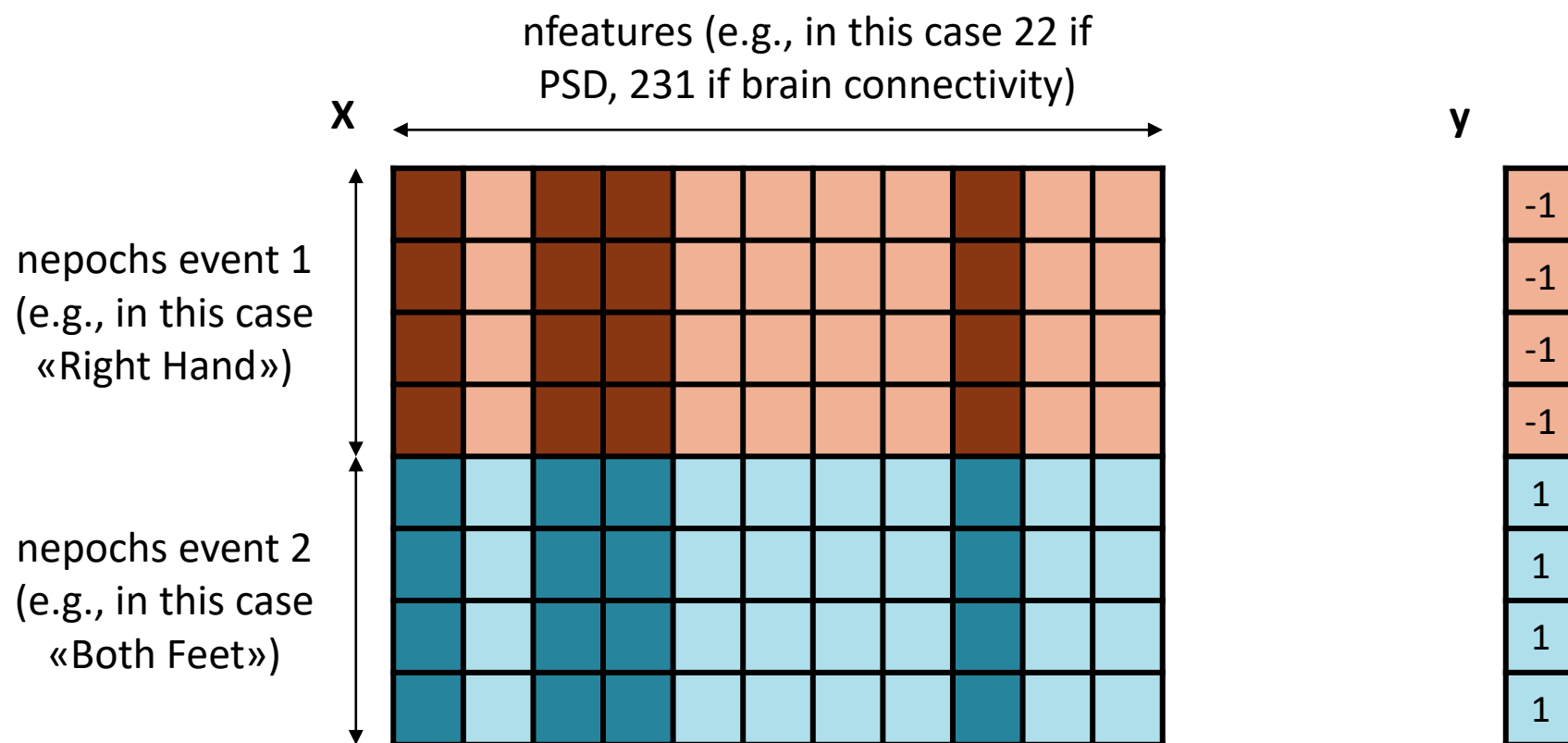
Main approaches:

- **Feature extraction** – transform original features in new ones based on their combination
 - E.g., Principal Component Analysis, Linear Discriminant Analysis, Multidimensional Scaling, etc.
- **Feature selection** – select a subset of the original features according to a criterion
 - *Filter methods*; they mainly act as rankers, depending on intrinsic properties of the data like variance, correlation, etc.
 - *Wrapper methods*; they use a ML algorithm to find the best subset of features
 - *Embedded methods*; they are a combination of the previous ones

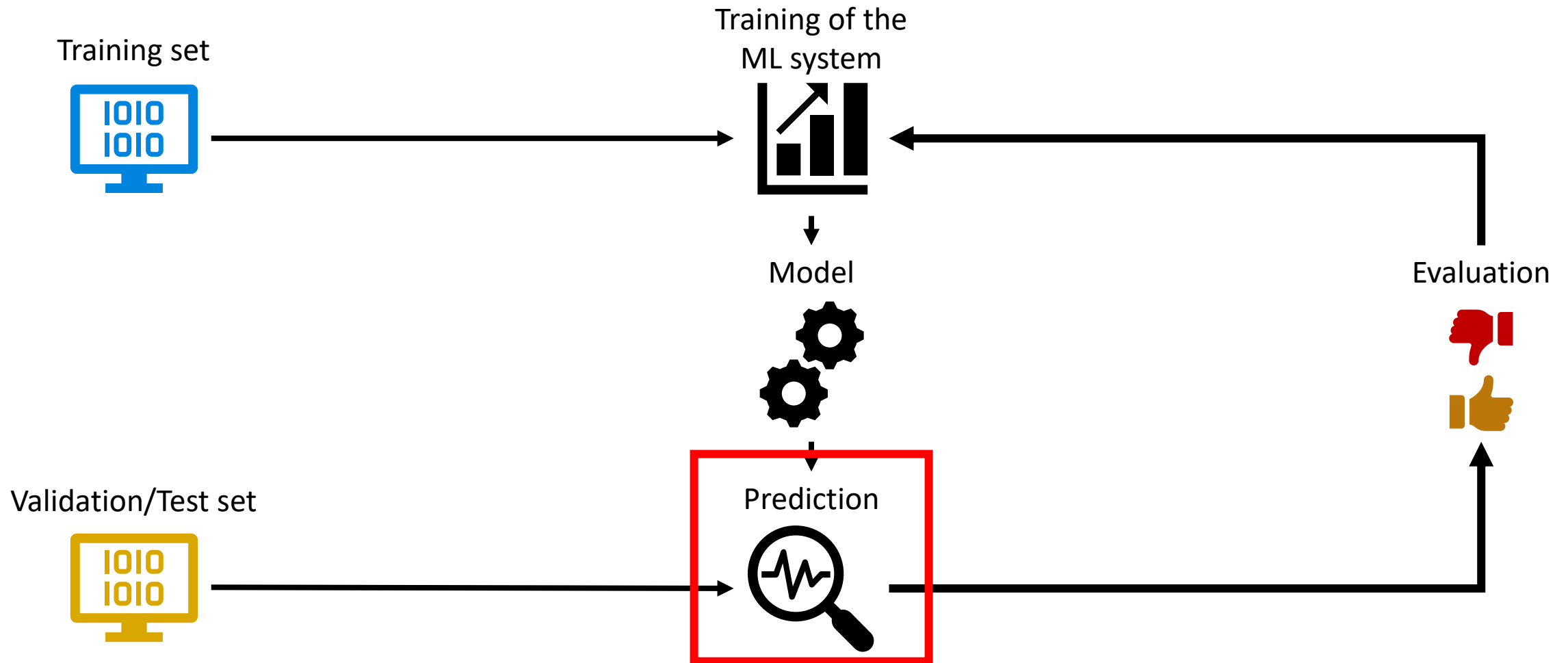
Evaluating the training: dimensionality reduction

- Fisher score can be used as a filter method
- According to this criterion, the best features are those with distances between data points in different classes as large as possible, and distances between data points in the same class as small as possible
- The Fisher score is usually evaluated for each feature individually
- $$F(X^{(j)}) = \frac{\sum_{k=1}^c n_k (\mu_k^{(j)} - \mu^{(j)})^2}{\sum_{k=1}^c n_k (\sigma_k^{(j)})^2}$$
 - j : j^{th} feature
 - c : classes
 - n_k : cardinality of the k th class
 - μ : mean
 - σ^2 : variance

Evaluating the training: dimensionality reduction



Machine Learning - a means for BCI



Classification performance measurements

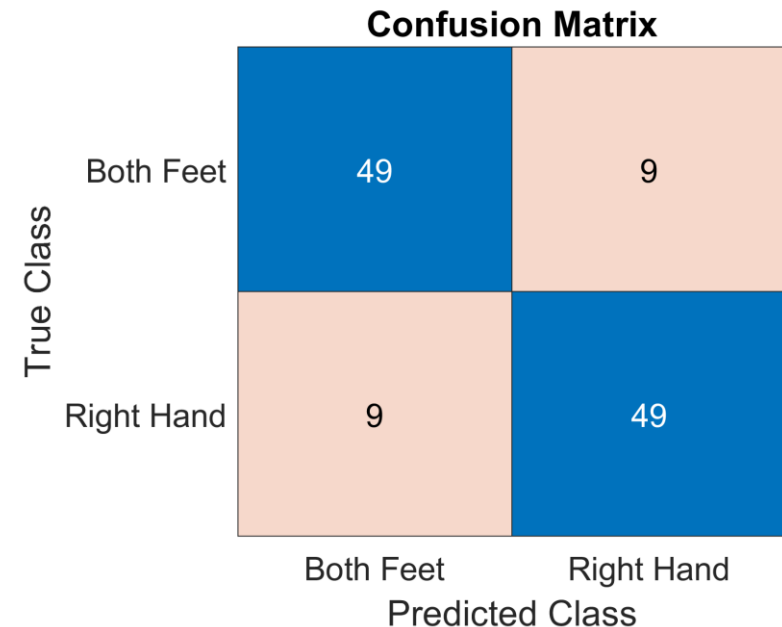
Confusion matrix: summary of the correctly and incorrectly classified samples; it can be easily generalized to the case of N classes

		ACTUAL VALUES		
		Positive	Negative	
PREDICTED VALUES	Positive	TP	FP	Type I error →
	Negative	FN	TN	
		↓ Type II error		

Classification performance measurements

```
% Extract the classification performance  
% measures from cross-validation  
confmat = confusionmat(y, predictions);  
% Plot the confusion matrix  
cm = confusionchart(confmat, ["Right Hand";  
"Both Feet"]);
```

- **y:** real labels of the samples
- **predictions:** predicted labels of the samples

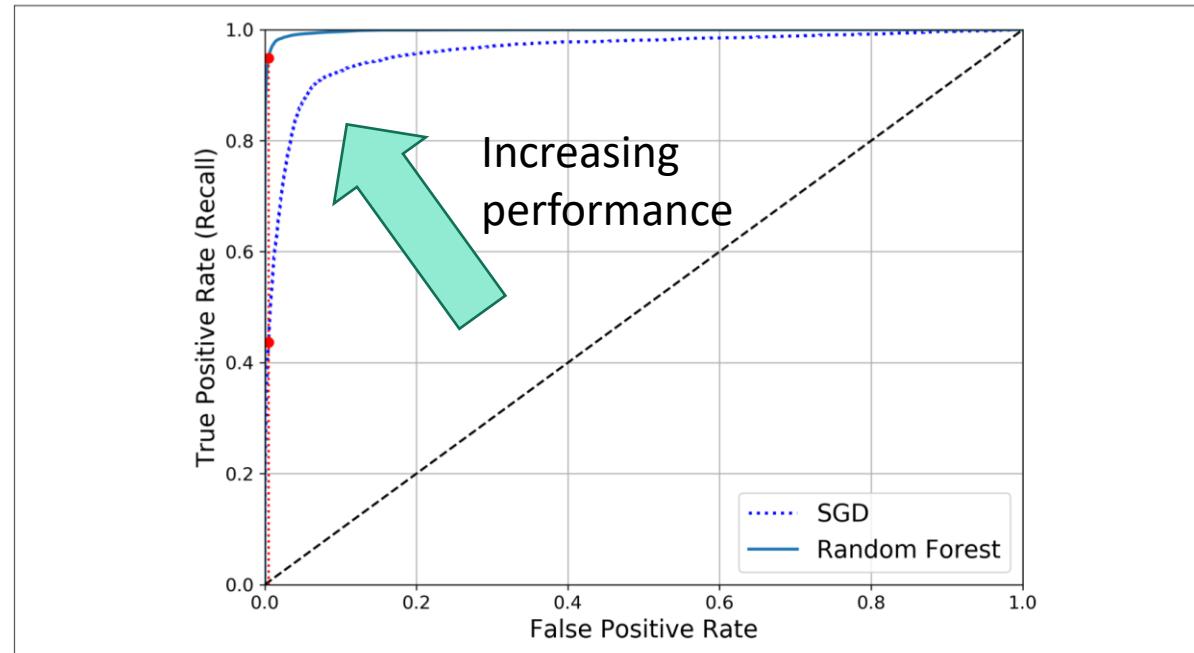


Classification performance measurements

- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$; this metric may be not reliable for imbalanced data
- **Precision:** $\frac{TP}{TP+FP}$
- **Recall/Sensitivity/True positive rate:** $\frac{TP}{TP+FN}$
- **Specificity/True negative rate:** $\frac{TN}{TN+FP}$
- **F₁ score:** harmonic mean of the precision and recall $2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$

Classification performance measurements

- **Receiver Operating Characteristic (ROC):** true positive rate against the false positive rate
- A perfect classifier has an **area under the curve (AUC)** equal to 1, whereas a purely random classifier has an AUC equal to 0.5

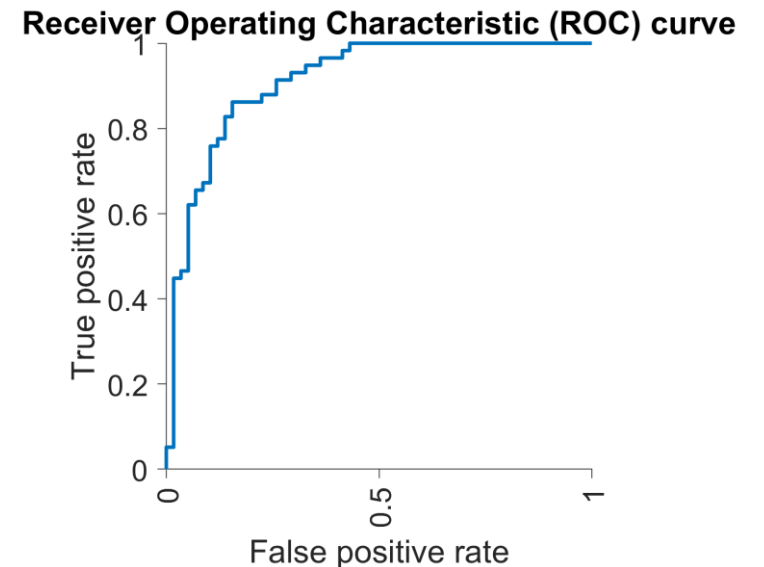


From «Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems» Aurélien Géron, 2nd Edition

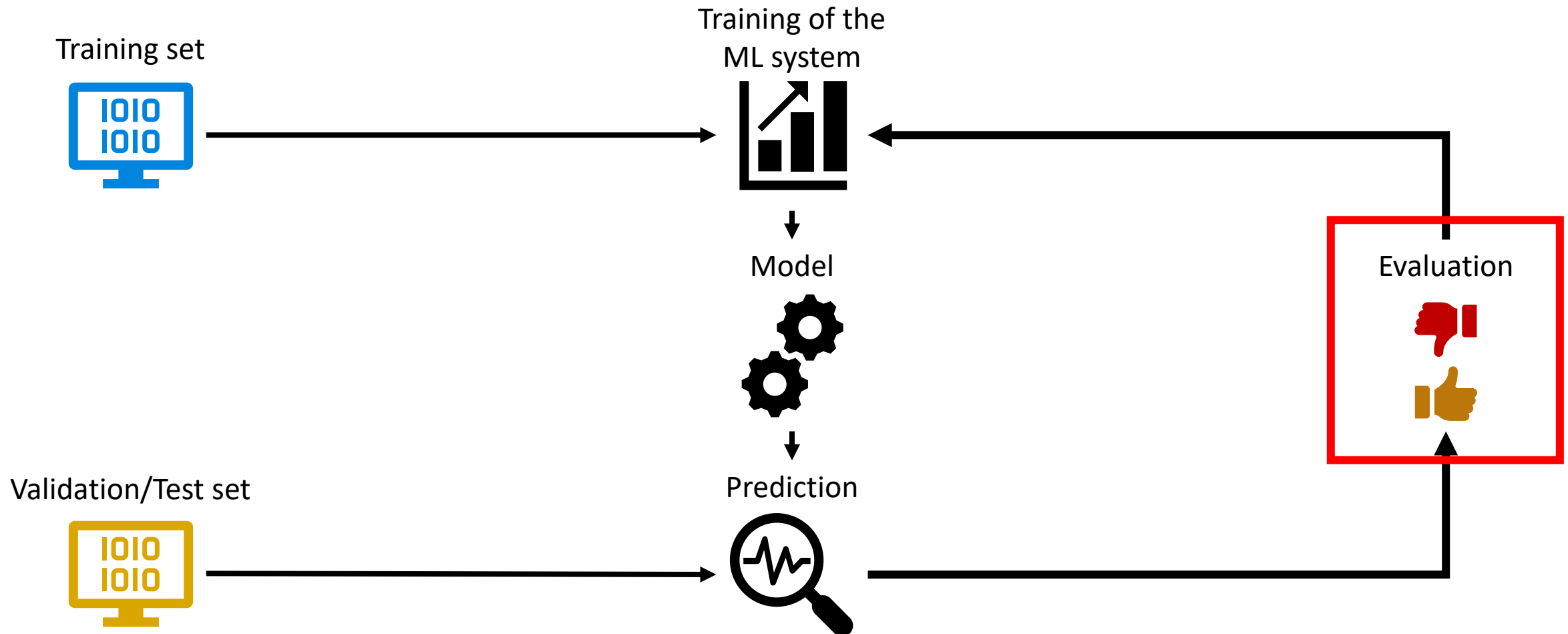
Classification performance measurements

```
% Calculate posterior probabilities or scores
% from a SVM cross-validated SVM model
[~, scores] = kfoldPredict(svm_cv);
% Or from a non-cross-validated SVM model
[~, scores] = predict(svm, X);
% Calculate FPR (ROC X) and TPR (ROC Y)
[X_ROC, Y_ROC] = perfcurve(y, scores(:,2), 1);
% Visualize the ROC curve
plot(X_ROC, Y_ROC);
```

- **scores(:,2):** the second column of scores contains the probabilities to belong to the second class (in this case: 1) for each observation
- **1:** specification of the positive class (in this case: 1)

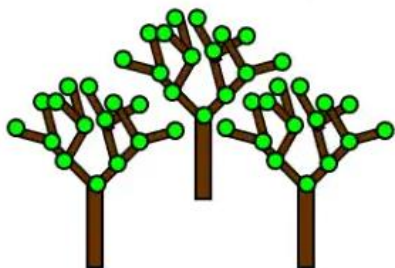


Machine Learning - a means for BCI

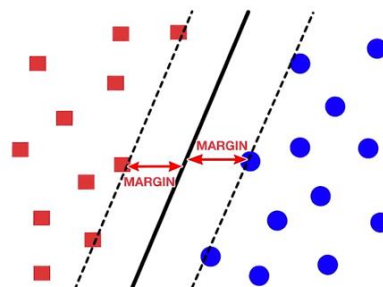


Testing

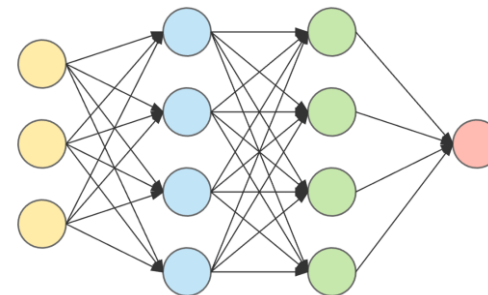
- **Generalization error/out-of-sample error:** error rate on the new samples of the test set
- Depending on the performance on new instances, it may be preferable to change the model



Random Forest

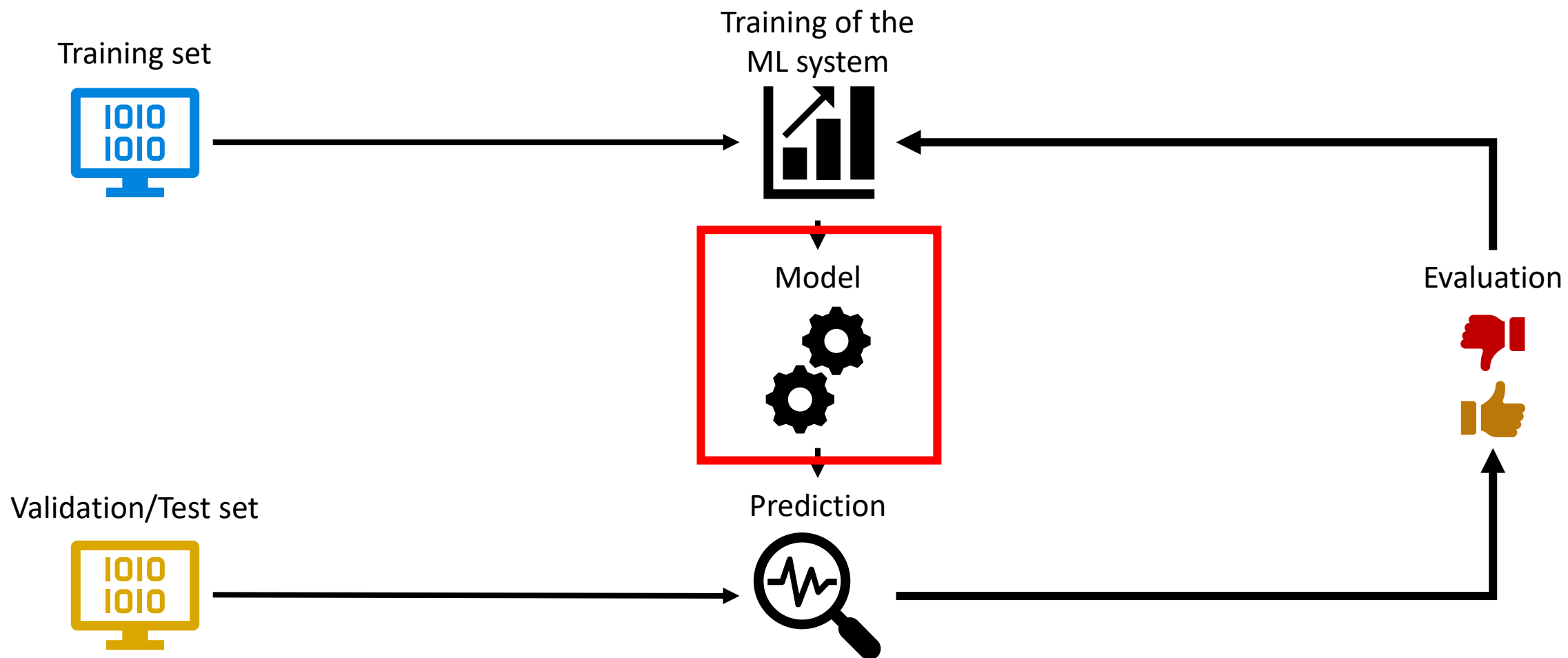


Support Vector
Machine



Artificial Neural
Network

Machine Learning - a means for BCI



BCI classifiers

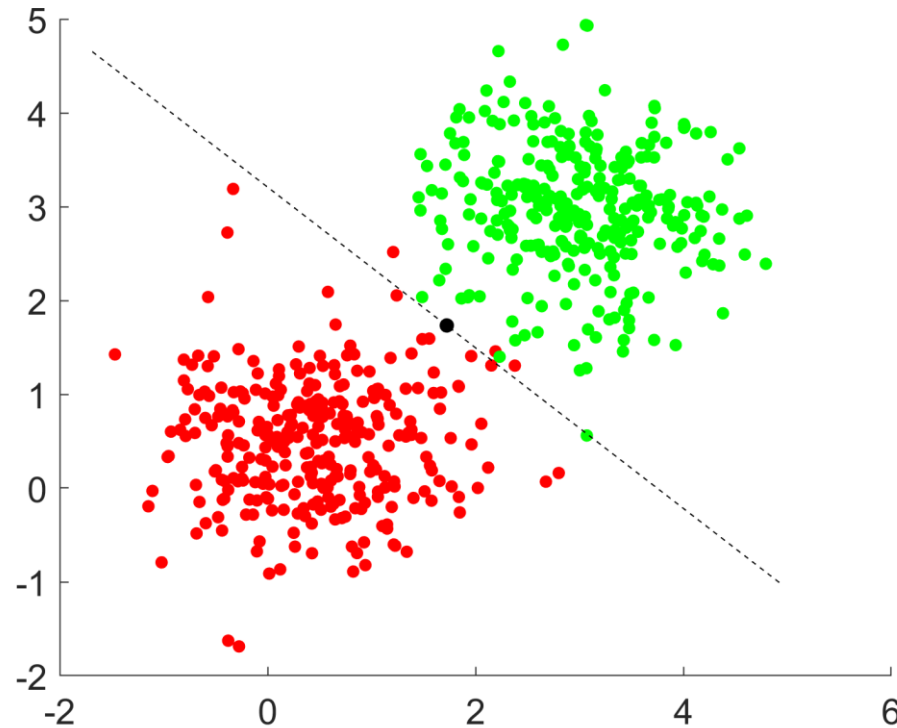
	Approach	Some properties
Generative model	Bayesian analysis	• Produces non-linear decision boundaries • Not very popular in BCI systems
Linear	Linear Discriminant Analysis (LDA)	• Simple classifier with acceptable accuracy • Low computation requirements • Usually two classes (multi-class versions exist) • Regularization required • Improved versions: Bayesian LDA and Fisher LDA
Non-linear	Support Vector Machine (SVM)	• Binary or multi-class • Regularization required • Speedy
	k-Nearest Neighbor Classifier (k-NNC)	• Multi-class • Efficient with low dimensional features
	Artificial Neural Network (ANN)	• Very flexible • Multi-class • Multiple architectures

BCI classifiers

	Approach	Some properties
Generative model	Bayesian analysis	• Produces non-linear decision boundaries • Not very popular in BCI systems
Linear	Linear Discriminant Analysis (LDA)	• Simple classifier with acceptable accuracy • Low computation requirements • Usually two classes (multi-class versions exist) • Regularization required • Improved versions: Bayesian LDA and Fisher LDA
Non-linear	Support Vector Machine (SVM)	• Binary or multi-class • Regularization required • Speedy
	k-Nearest Neighbor Classifier (k-NNC)	• Multi-class • Efficient with low dimensional features
	Artificial Neural Network (ANN)	• Very flexible • Multi-class • Multiple architectures

Linear Discriminant Analysis Classification

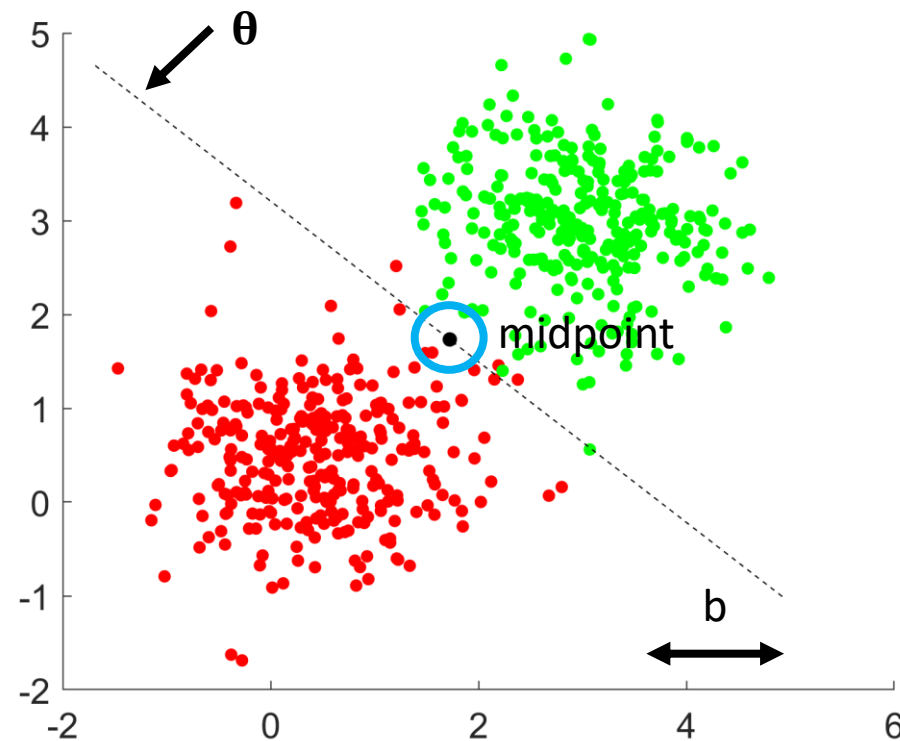
- During training, LDA learns the most discriminative axes between the classes
- The resulting axes can be used to define a hyperplane onto which project the data (dimensionality reduction)
- **Linear mapping:**
 $y = \theta x + b$
- **Classification:**
 $y = \text{sign}(\theta x + b)$



Linear Discriminant Analysis Classification

Given trial segments $\mathbf{x}_k$ (in vector form) in classes $\mathcal{C}_1$ and $\mathcal{C}_2$,

- $\boldsymbol{\mu}_i = \frac{1}{|\mathcal{C}_i|} \sum_{k \in \mathcal{C}_i} \mathbf{x}_k$
- $\Sigma_i = \sum_{k \in \mathcal{C}_i} (\mathbf{x}_k - \boldsymbol{\mu}_i)(\mathbf{x}_k - \boldsymbol{\mu}_i)^T$
- $\boldsymbol{\theta} = (\Sigma_1 + \Sigma_2)^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$
- $b = -\boldsymbol{\theta}^T(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)/2$



Linear Discriminant Analysis Classification

- **Assumptions**

- Data in each class is distributed according to a Gaussian distribution
- Shape of the distribution is identical for all classes

- **Benefits**

- Simple, fast and optimal in the large sample limit if assumptions are true

- **Problems**

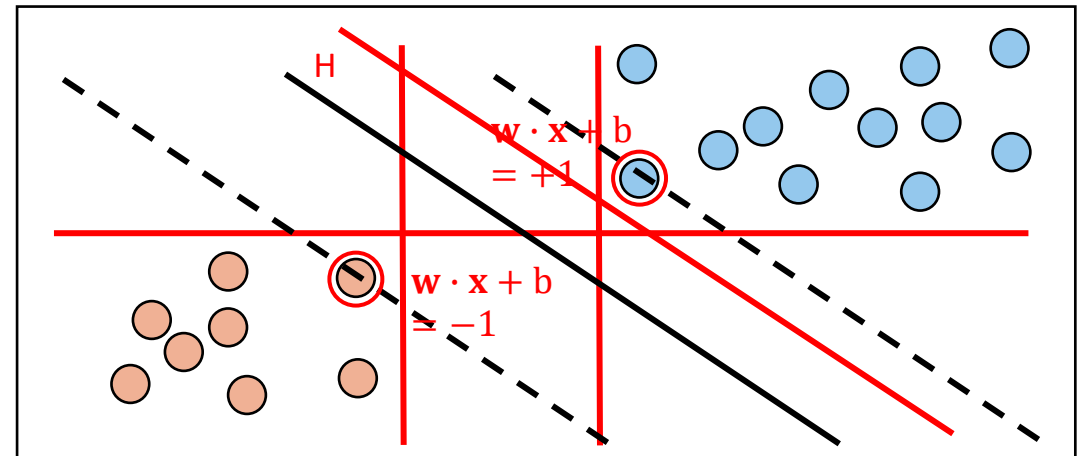
- Very sensitive to outliers (non-Gaussian)
- Covariance matrix estimates become unreliable/unusable for too few trials and too many dimensions

BCI classifiers

	Approach	Some properties
Generative model	Bayesian analysis	• Produces non-linear decision boundaries • Not very popular in BCI systems
Linear	Linear Discriminant Analysis (LDA)	• Simple classifier with acceptable accuracy • Low computation requirements • Usually two classes (multi-class versions exist) • Regularization required • Improved versions: Bayesian LDA and Fisher LDA
Non-linear	Support Vector Machine (SVM)	• Binary or multi-class • Regularization required • Speedy
	k-Nearest Neighbor Classifier (k-NNC)	• Multi-class • Efficient with low dimensional features
	Artificial Neural Network (ANN)	• Very flexible • Multi-class • Multiple architectures

Support Vector Machine: linear SVM

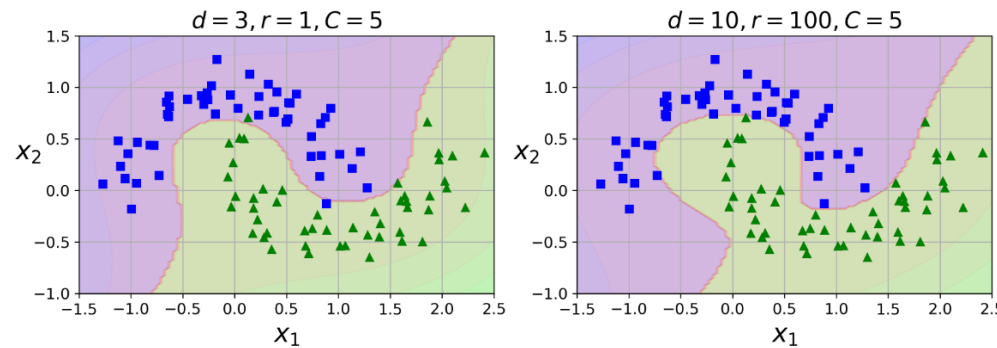
- A Support Vector Machine (SVM) is a supervised learning algorithm for classifying data
- It finds the **hyperplane** that best separates the data observations in the corresponding class
- A **linear** SVM finds the **linear combination of weights w** (one for each feature x) able to **predict y** (the class label) using the optimization criterion of **maximizing the margin**. Thus, the weights that are different from 0 are reduced to a few that correspond to the **support vectors**.
- Define the hyperplane H such that:
 - $w \cdot x_i + b \geq +1$ when $y = +1$
 - $w \cdot x_i + b \leq -1$ when $y = -1$



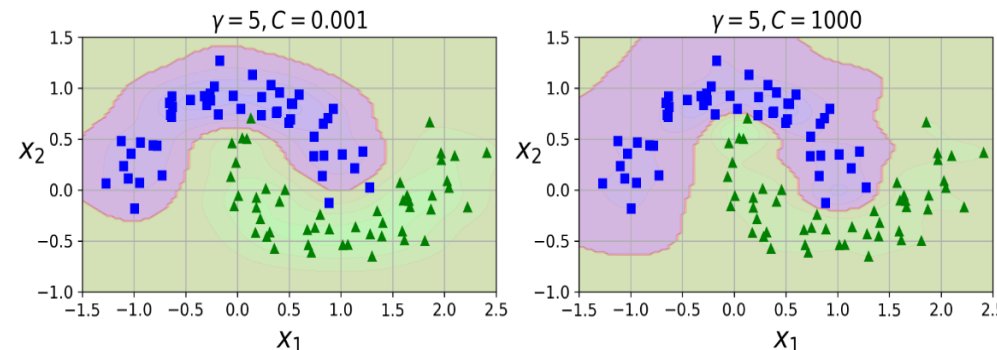
Support Vector Machine: kernel

The linear kernel is just one of the kernels that is possible to experiment with SVMs.
The **kernel** is the mapping of the observations into some feature space.
The principal kernels are:

- **Polynomial**



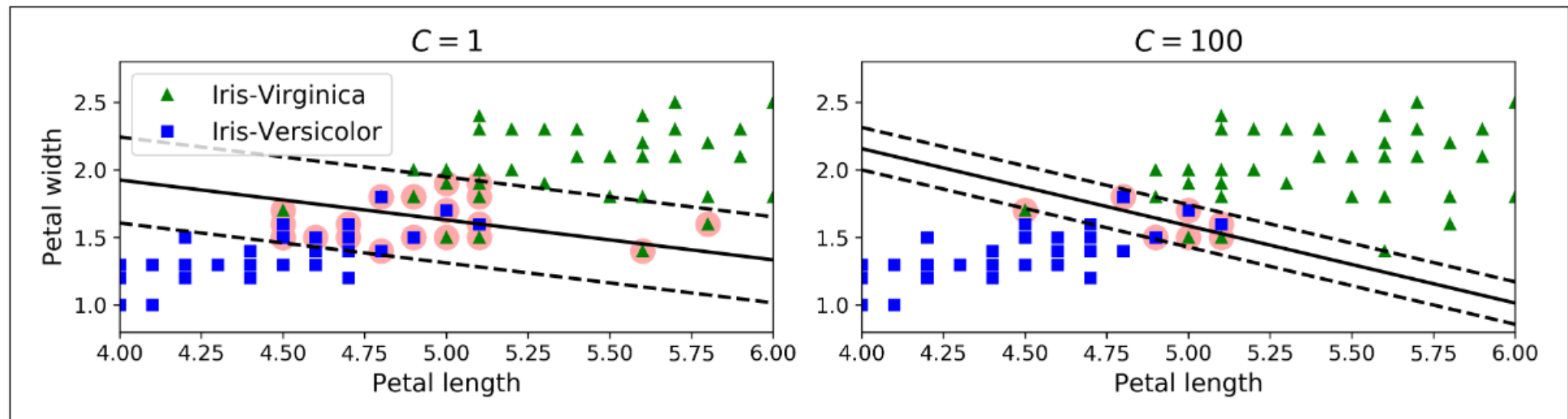
- **Radial**



From «Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems»
Aurélien Géron, 2nd Edition

Support Vector Machine: hyperparameter

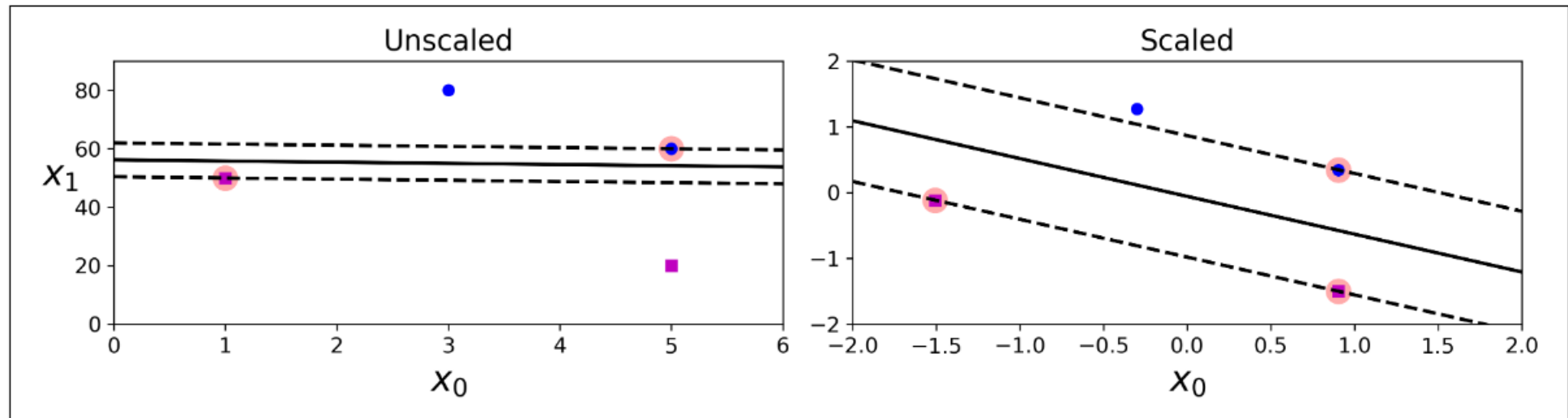
The main hyperparameter of the SVM is the **cost parameter C**, that is the penalization of the objective function for observations that cross the margin boundary for their class. Lower values of C allow the detection of larger margins for the decision boundary, while higher values of C allow the detection of smaller margins for the decision boundary.



From «Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems» Aurélien Géron, 2nd Edition

Support Vector Machine: feature scaling

Attention: SVMs are sensitive to the feature scales!



From «Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems» Aurélien Géron, 2nd Edition

Support Vector Machine: code

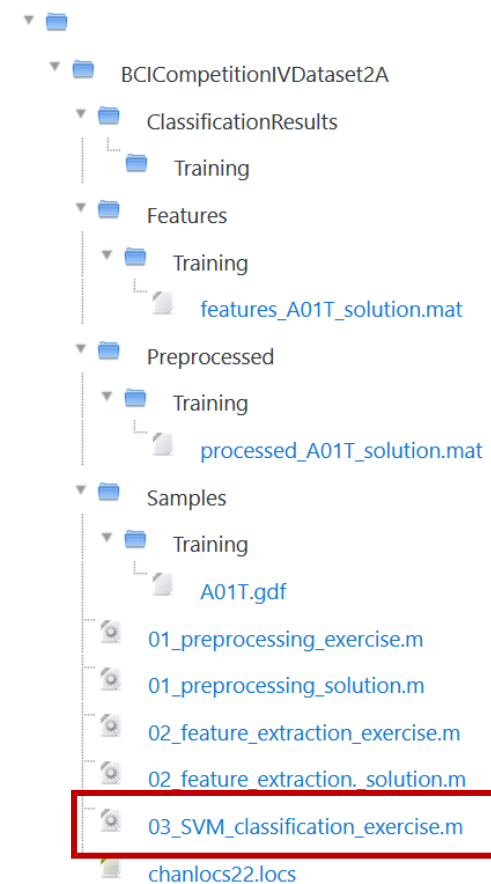
```
% Fit a SVM with cross-validation
svm_cv = fitcsvm(X_training, y_training, 'CVPartition', c_kfold, ...
    'KernelFunction', 'linear', ...
    'Standardize', true, ...
    'BoxConstraint', svm_optimization.ModelParameters.BoxConstraint, ...
    'KernelScale', svm_optimization.ModelParameters.KernelScale);
```

- **X_training**: observations (epochs x features)
- **y_training**: class labels (epochs x 1)
- **'CVPartition'**: perform cross-validation
- **'Standardize'**: center and scale each feature by the corresponding weighted column mean and standard deviation
- **'BoxConstraint'** and **'KernelScale'**: set the hyperparameters with values as resulted from the previous optimization

The **final model** can be obtained by fitting the SVM with the same hyperparameters on the whole training set (without cross-validation)

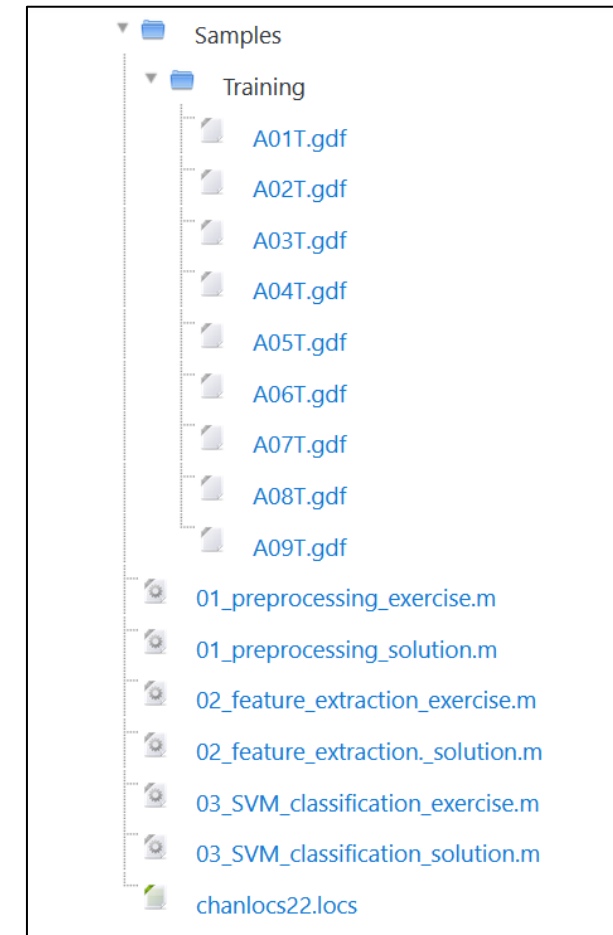
Exercise

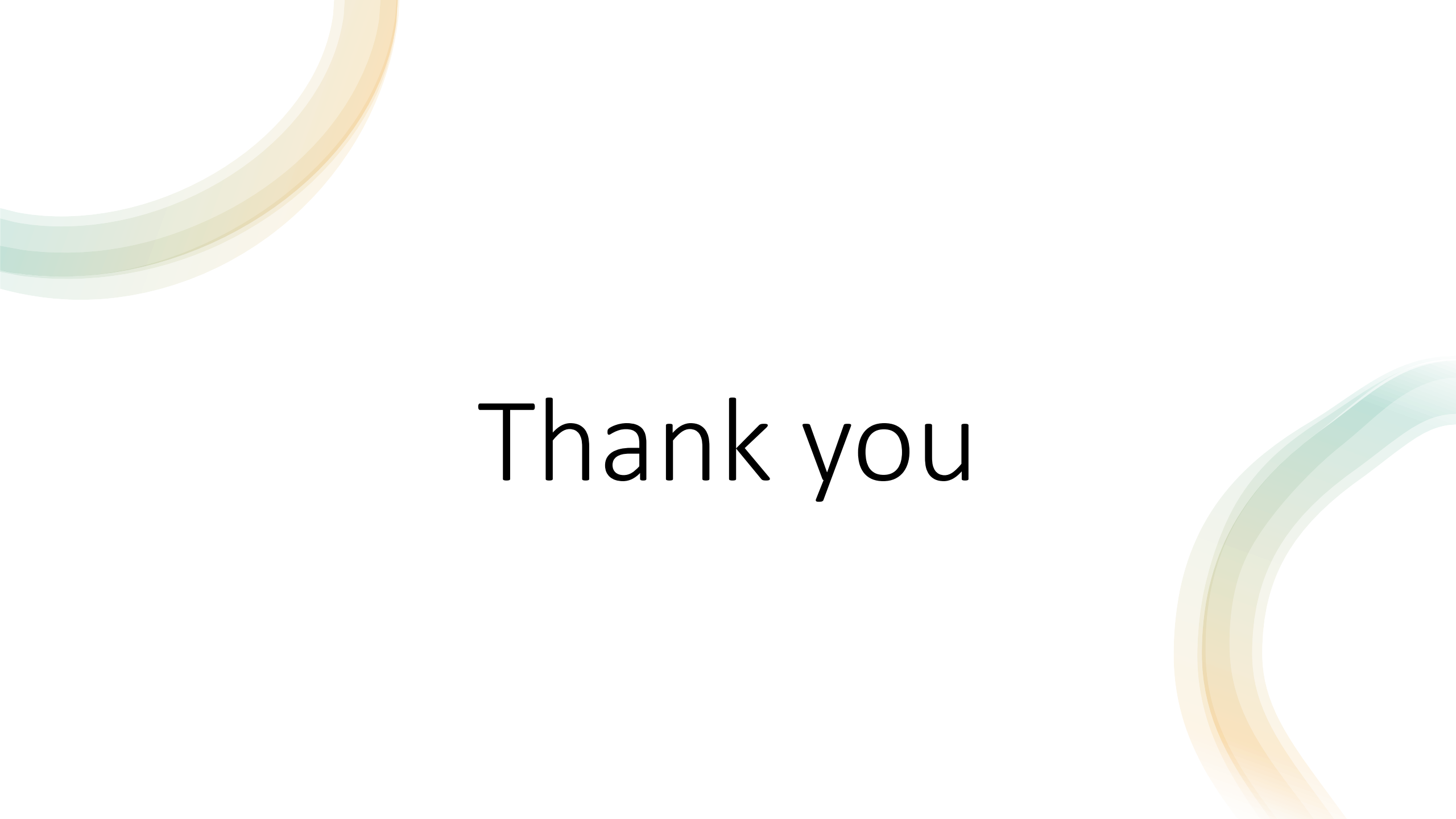
Hands-On



Final Evaluation

- Analyze a new subject from preprocessing to classification.
- Consider to use different features for classification (i.e., different frequency band or brain connectivity)
- Consider to explore a different combination of motor imagery tasks (e.g., Left Hand vs Tongue)



The image features a white background with two decorative curved lines. One line, in the top-left corner, is composed of three overlapping arcs in shades of yellow and green. Another similar line, also with three overlapping arcs in the same color palette, is located in the bottom-right corner. Centered between these decorative elements is the text "Thank you" in a black, sans-serif font.

Thank you